

From Lattices to Tensor Cores: Accelerating Private Information Retrieval

Sidaarth Sabhnani
UT Austin
ssabhnani@utexas.edu

David J. Wu
UT Austin
dwu4@cs.utexas.edu

Abstract

This work introduces SandwichPIR, the first single-server PIR protocol that implements the overwhelming majority of the server computation as dense 8-bit integer matrix multiplications on GPU tensor cores and requires no offline communication. For a 4 GB database with 32 KB records, SandwichPIR answers a query in 8.2 ms and communicates 688 KB of data. This amounts to a server throughput of 488 GB/s and is 88× faster than the best CPU-based protocol that does not rely on offline communication.

The performance of SandwichPIR shines when processing a batch of queries from many *independent* clients. This moves the server from a memory-bandwidth-bound regime into a compute-bound regime. A single Nvidia L40S GPU can process a batch of 128 queries to the same 4 GB database in 21.0 ms. This gives an amortized per-query processing time of 0.16 ms and an effective throughput of roughly 24 TB/s. This is 50× higher than the single-query throughput. With a fleet of 8 GPUs, SandwichPIR can process a batch of 128 queries to a 256 GB database in 119 ms and achieves an effective throughput of nearly 270 TB/s.

Finally, we show how to use SandwichPIR to enable private access to (text-only) English Wikipedia (8 GB compressed). Retrieving an article (up to 128 KB) requires 768 KB of client-server communication, with an estimated total end-to-end latency under 200 ms over a broadband network connection. By processing 64 queries at a time, a single GPU can handle over 2,500 queries per second (with a computational cost of \$0.20 per million queries based on current AWS pricing).

1 Introduction

Private information retrieval (PIR) [25, 46] allows a client to retrieve a record from a database without revealing to the server which record they want. The goal in PIR is to do so with communication that is sublinear in the database size. PIR has applications to private certificate transparency auditing [56, 43, 40, 63, 38, 50], credential breach reporting [76, 52, 5], private web search [39], and more. Companies like Apple [7, 8], Meta [26], and Google [83] have deployed PIR protocols to enable various privacy-preserving applications.

PIR protocols have been considered in many settings. In this work, we focus on the single-server model where the database is hosted on a single server. This is in contrast to multi-server PIR [25], where the database is replicated across multiple servers, and security relies on a more brittle non-collusion assumption. In addition, we focus on constructions in the silent preprocessing model, where there is no client-specific state or server-side hint that the client must upload or download prior to the protocol. As articulated in [59, §1.1], such hintless or stateless PIR protocols are desirable for applications where clients do not make enough queries to amortize the cost of the offline communication, where applications demand user anonymity, or when considering dynamic databases which may require frequent state updates.

All concretely-efficient single-server PIR protocols that support silent preprocessing require the server to perform a linear scan over the database [39, 51, 63, 20, 59]. For the fastest schemes today (i.e., those in the SimplePIR family [40]), existing implementations are mostly CPU-based and their throughput (i.e., the speed at which the server can process a query as a function of the database size) is constrained by the memory bandwidth of the machine. On modern CPUs, this is typically 10–20 GB/s per core. This directly limits the scalability of CPU-based PIR schemes to large databases.

Hardware acceleration for PIR. The primary operation in nearly all lattice-based PIR protocols is computing a matrix product between a client’s query vector and a database matrix. This makes lattice-based PIR schemes directly amenable to hardware acceleration. A recent line of works has shown how to accelerate lattice-based PIR schemes using GPUs [71, 50, 78, 42, 44] and even ASICs [45]. Most of these protocols leverage cross-client batching [56, 63, 50] where the server processes a batch of queries from multiple *independent* clients at once, and thus amortizes the cost of the linear scan over multiple queries. With batching, these recent works achieve very high effective server throughputs, ranging from 1–2 TB/s on a GPU [50, 42], to 14.5 TB/s using tensor cores on GPUs [44], and to nearly 23 TB/s on an ASIC [45]. However, these works all require offline communication. Some schemes require the client to upload a packing key that is several MB before making queries [71, 78, 42, 44]. Other schemes accelerate the SimplePIR protocol, which

requires the client to download a database-dependent hint (often several hundred MB) before making queries [50, 45].

Recent works [39, 51, 63, 59] have shown how to combine SimplePIR with a hint-compression algorithm to obtain a scheme with comparable efficiency, but no offline communication. However, a priori, these approaches do not appear directly amenable to hardware acceleration. Indeed, the work of [50] implements the hint compression on the CPU, and compared to the GPU-accelerated SimplePIR component, the CPU portion comprises over 99% of the total server processing time. The work of [44] develops a new GPU-friendly packing scheme to combine with SimplePIR, which in turn requires the client to upload a packing key (≈ 5 MB). In fact, [44, §3.5.3] explicitly says that the hint-compression approach from schemes like HintlessPIR [51] and InsPIRe [59] exhibits inherent limitations on GPUs due to long sequential loops arising from data-dependent transformations.

This work. Our main technical contribution is showing that the hint-compression algorithm from InsPIRe [59] can in fact be written as a dense matrix multiplication between a vector from the client’s query and a query-independent matrix that the server can compute ahead of time (Section 3.1). Combined with the fact that the server computation in the base SimplePIR protocol is also a matrix multiplication between the client’s query and the database matrix, this means we can recast the entirety of InsPIRe¹ as computing essentially two matrix multiplications. In fact, the algebraic structure of the InsPIRe hint-compression algorithm even allows these two matrix multiplications to proceed *concurrently* (i.e., hint compression can occur in parallel with the linear scan). In combination with a modulus switching idea (Section 3.2), we obtain our SandwichPIR protocol, a new variant of InsPIRe tailored for GPU acceleration.

Similar to [44] (concurrent with this work), we design SandwichPIR to take advantage of tensor cores on modern Nvidia GPUs. The main computation in SandwichPIR is the SimplePIR linear scan, which essentially performs a single multiply-accumulate operation per byte of the database. This is precisely the type of workflow favored by tensor cores (which can be thought of as small units for multiplying matrices on 8-bit inputs and accumulating the output in 32-bit registers). On a modern GPU such as the Nvidia L40S, tensor cores can sustain 733 trillion operations per second (TOPS). In comparison, a standard AWS instance like the `r7i.2xlarge` with 4 Intel Sapphire Rapids cores peaks at 3.3 TOPS (or 26.2 TOPS with Intel AMX). Based on current AWS pricing, the GPU delivers 8 $\times$ more compute and 4 $\times$ more memory bandwidth than the CPU instance per dollar

(cf. Table 4). We design SandwichPIR to take full advantage of tensor cores. To our knowledge, SandwichPIR is the first GPU-based single-server PIR protocol that does not require any offline communication.

Summary of results. On a single Nvidia L40S, SandwichPIR answers a query to a 4 GB database with 32 KB records in 8 ms, using 688 KB of communication and no offline communication. This corresponds to a single-query throughput of 488 GB/s, which is 88 $\times$ higher than that of InsPIRe [59], the fastest CPU protocol with silent preprocessing. With cross-client batching, the effective throughput on a single Nvidia L40S peaks at 24.5 TB/s, and corresponds to an amortized 0.16 ms per query. This matches what a custom ASIC is able to achieve [45] and is faster than the VIPIR scheme, which peaks at 14.5 TB/s on a more powerful GPU (an Nvidia H100) [44, Table 3]. Moreover, our protocol achieves the higher throughput *without* any offline communication. The ASIC-based version of SimplePIR from [45] requires large database-dependent hints while the VIPIR protocol [44] requires the client to upload a public key (several MB) before issuing queries. Using a fleet of 8 GPUs, SandwichPIR can answer a batch of 128 queries to a 256 GB database in 119 ms, which translates to an effective throughput of nearly 270 TB/s. We refer to Section 5 for the full breakdown and evaluation.

Finally, as a proof of concept, we show how to use SandwichPIR to provide private access to (text-only) English Wikipedia (8 GB compressed). Here, a single GPU serves over 2,500 queries per second at a computational cost of \$0.20 per million queries, and we estimate that a client can fetch an article with an end-to-end latency around 151 ms over a broadband connection and 768 KB of total communication. We describe this application in Section 5.2.

2 Preliminaries

Throughout this work, we write λ to denote the security parameter. For a positive integer $n \in \mathbb{N}$, we write $[n]$ to denote the set $[n] := \{1, \dots, n\}$. For integers $a < b$, we write $[a, b] := \{a, a + 1, \dots, b\}$. For a positive integer $q \in \mathbb{N}$, we write $\mathbb{Z}_q$ for the integers modulo q . We use bold uppercase letters (e.g., $\mathbf{A}, \mathbf{B}$) to denote matrices and bold lowercase letters (e.g., $\mathbf{u}, \mathbf{v}$) to denote vectors. We write $\mathbf{A}^\top$ to denote the transpose of $\mathbf{A}$. By default, all vectors are column vectors and we write row vectors as transposes (e.g., $\mathbf{u}^\top$). For a finite set S , we write $x \stackrel{\mathcal{R}}{\leftarrow} S$ to denote that x is sampled uniformly from the set S . We say an algorithm is efficient if it runs in probabilistic polynomial time in the length of its input. We write $\text{negl}(\lambda)$ to denote a function that is $o(\lambda^{-c})$ for all $c \in \mathbb{N}$ and $\text{poly}(\lambda)$ to denote a function that is $O(\lambda^c)$ for some constant $c \in \mathbb{N}$. We say two ensembles of distributions $\mathcal{D}_0 = \{\mathcal{D}_{0,\lambda}\}_{\lambda \in \mathbb{N}}$ and $\mathcal{D}_1 = \{\mathcal{D}_{1,\lambda}\}_{\lambda \in \mathbb{N}}$ are computationally

¹Strictly speaking, we consider the most basic version of InsPIRe that composes SimplePIR with the InsPIRe ring packing procedure. The full InsPIRe protocol also leverages homomorphic polynomial evaluation to further reduce the online communication. We do not consider that in this work. We refer to Section 3.2 for the precise details.

indistinguishable if no efficient adversary can distinguish between them except with negligible probability.

Private information retrieval. We now recall the formal definition of private information retrieval (PIR) [46]. Following [40, 51, 63, 59], we consider the setting where we allow for a database-dependent preprocessing step that outputs a set of short *public* parameters pp together with an encoded database edb that the server will use to respond to queries. The client uses the public parameters to issue queries.

Definition 2.1 (PIR with Preprocessing). Let λ be a security parameter and $N = N(\lambda)$ be the database size. A (two-message) single-server private information retrieval scheme with preprocessing Π_{PIR} with message space $\mathbb{Z}_p$ is a tuple of efficient algorithms $\Pi_{\text{PIR}} = (\text{Setup}, \text{Query}, \text{Answer}, \text{Recover})$ with the following syntax:

- $\text{Setup}(1^\lambda, db) \rightarrow (pp, edb)$: On input the security parameter λ and a database $db \in \mathbb{Z}_p^N$, the setup algorithm outputs a set of public parameters pp and an encoded database edb .
- $\text{Query}(pp, i) \rightarrow (qu, st)$: On input the public parameters pp and an index $i \in [N]$, the query algorithm outputs a query qu and some state st .
- $\text{Answer}(edb, qu) \rightarrow ans$: On input the encoded database edb , and a query qu , the answer algorithm outputs an answer ans .
- $\text{Recover}(st, ans) \rightarrow r$: On input a state st and an answer ans , the recover algorithm outputs a message $r \in \mathbb{Z}_p$.

Moreover, we require the above algorithms to satisfy the following properties:

- **Correctness:** For all security parameters $\lambda \in \mathbb{N}$, all databases $db \in \mathbb{Z}_p^N$, all (pp, edb) in the support of $\text{Setup}(1^\lambda, db)$, and all indices $i \in [N]$, we have that

$$\Pr[\text{Recover}(st, ans) = db_i] = 1, \quad (2.1)$$

where $(qu, st) \leftarrow \text{Query}(pp, i)$ and $ans \leftarrow \text{Answer}(edb, qu)$. We say that the scheme has correctness error at most $\delta = \delta(\lambda)$ if the probability in Eq. (2.1) is at least $1 - \delta$.

- **Query privacy:** For a security parameter λ and a bit $b \in \{0, 1\}$, we define the query-privacy game between an adversary $\mathcal{A}$ and a challenger as follows:
 - On input the security parameter 1^λ , the adversary $\mathcal{A}$ outputs a database $db \in \mathbb{Z}_p^N$.
 - The challenger computes and sends $(pp, edb) \leftarrow \text{Setup}(1^\lambda, db)$ to $\mathcal{A}$.
 - Algorithm $\mathcal{A}$ outputs a pair of indices $i_0, i_1 \in [N]$.
 - The challenger computes $(qu, st) \leftarrow \text{Query}(pp, i_b)$ and gives the query qu to $\mathcal{A}$.
 - Algorithm $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$, which is the output of the experiment.

The scheme satisfies query privacy if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[b' = 1 : b = 0] - \Pr[b' = 1 : b = 1]| = \text{negl}(\lambda).$$

- **Succinctness:** There exists a polynomial $\text{poly}(\cdot)$ such that for all security parameters $\lambda \in \mathbb{N}$, all polynomials $N = N(\lambda)$, all databases $db \in \mathbb{Z}_p^N$, all (pp, edb) in the support of $\text{Setup}(1^\lambda, db)$, all indices $i \in [N]$, all queries (qu, st) in the support of $\text{Query}(pp, i)$, and all responses ans in the support of $\text{Answer}(edb, qu)$, we have that $|pp| + |qu| + |ans| = o(N) \cdot \text{poly}(\lambda)$.

2.1 Lattice Preliminaries

We now recall some key facts about lattice-based cryptography that will be relevant to our design.

Discrete Gaussians. For a width parameter $\sigma > 0$, the discrete Gaussian distribution $D_{\mathbb{Z}, \sigma}$ over $\mathbb{Z}$ with mean 0 and width σ has probability mass function $\Pr[x] \propto \exp(-\pi x^2 / \sigma^2)$.

Ring LWE. Like many prior works on PIR (cf. [61, 6, 30, 65, 62, 39, 51, 63, 50, 59]), we work over the cyclotomic ring $R = \mathbb{Z}[x]/(x^n + 1)$, where n is a power-of-two. Throughout this paper, we write $R_q := R/qR = \mathbb{Z}_q[x]/(x^n + 1)$, where $q \in \mathbb{N}$ is a positive integer. We write $D_{R, \sigma}$ to denote the distribution over R where each coefficient of the polynomial is sampled independently from the discrete Gaussian distribution $D_{\mathbb{Z}, \sigma}$. The ring learning with errors (RLWE) assumption [58] with parameters (n, m, q, σ) asserts that for $\mathbf{a} \xleftarrow{R} R_q^m$, $s \xleftarrow{R} R_q$, $\mathbf{e} \leftarrow D_{R, \sigma}^m$, and $\mathbf{v} \xleftarrow{R} R_q^m$, the distributions $(\mathbf{a}, \mathbf{sa} + \mathbf{e})$ and $(\mathbf{a}, \mathbf{v})$ are computationally indistinguishable. Like previous works, we will use the normal form of RLWE, where the secret s is sampled from the error distribution (i.e., $s \leftarrow D_{R, \sigma}$). The work of [9] shows that the normal-form variant of the problem is as hard as RLWE with uniform secrets.

Rounding. For $x \in \mathbb{R}$, we write $\lfloor x \rfloor$ to denote the nearest integer to x . We write $\lfloor x \rfloor$ and $\lceil x \rceil$ to denote the floor and ceiling functions, respectively. For integers q and q' , we write $\lfloor \cdot \rfloor_{q, q'} : \mathbb{Z}_q \rightarrow \mathbb{Z}_{q'}$ to denote the function that lifts the input $x \in \mathbb{Z}_q$ to the representative $x' \in (-q/2, q/2]$ and outputs $\lfloor (q'/q) \cdot x' \rfloor$ in $\mathbb{Z}_{q'}$, where the division is performed over the rationals. We extend these operators to operate coefficient-wise on polynomials and component-wise on matrices.

Gadget decomposition. For a modulus $q \in \mathbb{N}$ and a decomposition base $z \in \mathbb{N}$, the base- z gadget vector [64] is $\mathbf{g}_z^T = [1, z, z^2, \dots, z^{t-1}]$ where $t = \lceil \log_z q \rceil$. We write $\mathbf{g}_z^{-1} : R_q \rightarrow R^t$ to denote the base- z decomposition operator that acts coefficient-wise on a polynomial $a \in R_q$, expanding

each coefficient into its base- z representation with digits in $(-z/2, z/2]$, such that $\mathbf{g}_z^\top \cdot \mathbf{g}_z^{-1}(a) = a$ for all $a \in R_q$.

LWE and RLWE ciphertexts. Let p be the plaintext modulus and $\Delta = \lfloor q/p \rfloor$ be a scaling factor. We say a pair $(a, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ is an LWE encryption of m with respect to a secret key $\mathbf{s}$ if $b = -\mathbf{s}^\top \mathbf{a} + e + \Delta \cdot m$, for some small error $e \in \mathbb{Z}$. Similarly, we say a pair $(a, b) \in R_q^n \times R_q$ is an RLWE encryption of $m \in R_p$ with respect to a secret key $\mathbf{s} \in R$ if $b = -\mathbf{s}a + e + \Delta \cdot m$ for some small error $e \in R$ (i.e., a polynomial with small coefficients). Observe that given the ciphertext (a, b) and the secret key $\mathbf{s}$, one can decrypt by computing $\lfloor b + \mathbf{s}a \rfloor_{q,p}$ and recover the message m . We refer to a as the message-independent component of the ciphertext and b as the message-dependent component.

Key switching. A standard technique in homomorphic encryption [14, 13] is the ability to take an RLWE ciphertext $c = (a, b)$ encrypting a message m under the secret key $\mathbf{s}$ and convert it into a new ciphertext $c' = (a', b')$ encrypting the same message m under a new key $\mathbf{s}'$. To do so, we publish a set of key-switching parameters, which is essentially an encryption of $\mathbf{s}$ under $\mathbf{s}'$. Concretely, the key-switching key is the pair $(\mathbf{a}_{\text{ks}}^\top, \mathbf{b}_{\text{ks}}^\top)$, where $\mathbf{a}_{\text{ks}}^\top \xleftarrow{R} R_q^t$ and $\mathbf{b}_{\text{ks}}^\top = -\mathbf{s}'^\top \mathbf{a}_{\text{ks}}^\top + \mathbf{e}_{\text{ks}}^\top + \mathbf{s} \cdot \mathbf{g}_z^\top \in R_q^t$, $\mathbf{e}_{\text{ks}} \leftarrow D_{R,\sigma}^t$, and $\mathbf{g}_z \in R_q^t$ is the base- z gadget vector defined above. We will often refer to the pair $(\mathbf{a}_{\text{ks}}^\top, \mathbf{b}_{\text{ks}}^\top)$ as a key-switching key with decomposition base z and error sampled from a discrete Gaussian distribution of width σ . Then, given $c = (a, b)$, the key-switched ciphertext $c' = (a', b')$ is

$$c' = (a', b') = (\mathbf{a}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a), b + \mathbf{b}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a)).$$

To see that c' is an encryption of m under $\mathbf{s}'$, observe that

$$\begin{aligned} b' + \mathbf{s}'^\top a' &= b + \mathbf{b}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a) + \mathbf{s}'^\top \mathbf{a}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a) \\ &= b + (-\mathbf{s}'^\top \mathbf{a}_{\text{ks}}^\top + \mathbf{e}_{\text{ks}}^\top + \mathbf{s} \cdot \mathbf{g}_z^\top) \mathbf{g}_z^{-1}(a) + \mathbf{s}'^\top \mathbf{a}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a) \\ &= b + \mathbf{s}a + \mathbf{e}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a) \\ &= \Delta \cdot m + e + \mathbf{e}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a), \end{aligned}$$

where e is the original error in the ciphertext c . Thus, c' is an encryption of m under $\mathbf{s}'$ with slightly larger error $e + \mathbf{e}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a)$. The key-switching operation has two appealing features that are exploited by modern PIR schemes:

- **Small noise growth:** The error introduced by key switching is *additive* and does not depend on the error in the input ciphertext c .
- **Support for preprocessing:** The first component of the output ciphertext $c' = \mathbf{a}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a)$ only depends on the message-independent component a of the input ciphertext. In schemes like [51, 59], the ring elements $\mathbf{a}_{\text{ks}}^\top$ and a are fixed as part of the public parameters, which means the product $\mathbf{a}_{\text{ks}}^\top \mathbf{g}_z^{-1}(a)$ can be computed in the *offline* stage. This is an important optimization to reduce the online cost of the PIR scheme.

Automorphisms. For an integer $\ell \in \mathbb{Z}_{2n}^*$, let $\tau_\ell: R_q \rightarrow R_q$ be the ring automorphism $r(x) \mapsto r(x^\ell)$. Given an RLWE ciphertext $c = (a, b)$ encrypting a message $m \in R_p$ under a secret key $\mathbf{s} \in R_q$, we define $\tau_\ell(c) := (\tau_\ell(a), \tau_\ell(b))$. Since τ_ℓ is a ring homomorphism, it is easy to see that $\tau_\ell(c)$ is an encryption of $\tau_\ell(m)$ under the key $\tau_\ell(\mathbf{s})$. Namely,

$$\tau_\ell(b) + \tau_\ell(\mathbf{s}) \cdot \tau_\ell(a) = \tau_\ell(b + \mathbf{s}a) = \tau_\ell(\Delta \cdot m + e) = \Delta \cdot \tau_\ell(m) + \tau_\ell(e),$$

where e is the error associated with the ciphertext c . Importantly, an automorphism does *not* increase the magnitude of the error in the ciphertext. To obtain an encryption of $\tau_\ell(m)$ under the *original* key $\mathbf{s}$, we use key-switching to switch from an encryption under $\tau_\ell(\mathbf{s})$ to an encryption under $\mathbf{s}$. We write $(\mathbf{a}_{\text{ks},\tau_\ell}^\top, \mathbf{b}_{\text{ks},\tau_\ell}^\top)$ for the associated key-switching parameters.

2.2 SimplePIR, YPIR, and InsPIRe

Our main contribution in this work is showing how to leverage modern GPUs to accelerate state-of-the-art single-server PIR protocols. We focus on the SimplePIR scheme [40] and specifically its hintless variants: YPIR [63] and InsPIRe [59]. In this section, we recall the core structure of these schemes.

Starting point: SimplePIR. In SimplePIR [40], the database is a matrix $\mathbf{D} \in \mathbb{Z}^{\ell_1 \times \ell_2}$, where each element is an integer in $(-p/2, p/2]$. Namely, each database component is a $\mathbb{Z}_p$ element lifted to the integers. SimplePIR works over integer lattices. In the following description, let n be the lattice dimension, q be the modulus, and σ be the width of a discrete Gaussian error distribution. The scheme works as follows:

- **Setup:** The public parameters consist of a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times \ell_1}$. In the setup phase, the server precomputes the hint $\mathbf{H} = \mathbf{A} \cdot \mathbf{D} \in \mathbb{Z}_q^{n \times \ell_2}$ and sends $\mathbf{H}$ to the client in the offline phase.
- **Client's query:** In SimplePIR, the client retrieves a row of the database. If the client is interested in row $i \in [\ell_1]$ of the database, then their query is the vector $\mathbf{q}^\top = -\mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top + \Delta \mathbf{u}_i^\top \in \mathbb{Z}_q^{\ell_1}$, where $\mathbf{s} \xleftarrow{R} \mathbb{Z}_q^n$ is an LWE secret, $\mathbf{e} \leftarrow D_{\mathbb{Z},\sigma}^{\ell_1}$ is the LWE error, $\mathbf{u}_i \in \mathbb{Z}^{\ell_1}$ is the indicator vector that is 1 at index $i \in [\ell_1]$ and 0 elsewhere, and $\Delta = \lfloor q/p \rfloor$ is the scaling factor. The client's state is the vector $\mathbf{s}$.
- **Server response:** The server sends back $\mathbf{b}^\top = \mathbf{q}^\top \mathbf{D} \in \mathbb{Z}_q^{\ell_2}$.
- **Answer decoding:** To recover the message, the client computes and outputs $\lfloor \mathbf{b}^\top + \mathbf{s}^\top \mathbf{H} \rfloor_{q,p} \in \mathbb{Z}_p^{\ell_2}$.

To see that this satisfies correctness, observe that

$$\begin{aligned} \mathbf{b}^\top + \mathbf{s}^\top \mathbf{H} &= \mathbf{q}^\top \mathbf{D} + \mathbf{s}^\top \mathbf{A} \mathbf{D} \\ &= -\mathbf{s}^\top \mathbf{A} \mathbf{D} + \mathbf{e}^\top \mathbf{D} + \lfloor q/p \rfloor \cdot \mathbf{u}_i^\top \mathbf{D} + \mathbf{s}^\top \mathbf{A} \mathbf{D} \\ &= \lfloor q/p \rfloor \cdot \mathbf{d}_i^\top + \mathbf{e}^\top \mathbf{D}, \end{aligned}$$

where $\mathbf{d}_i^\top \in \mathbb{Z}_p^{\ell_2}$ is the i^{th} row of the database. If $\|\mathbf{e}^\top \mathbf{D}\|$ is small (relative to Δ), then rounding recovers the row $\mathbf{d}_i^\top \in \mathbb{Z}_p^{\ell_2}$.

In SimplePIR, the server computation consists of a matrix-vector multiplication $\mathbf{q}^\top \mathbf{D}$ between the client's query and the database matrix. When multiple (independent) clients submit queries, the server can batch process these queries with a single (dense) matrix multiplication. The moduli p, q can also be chosen to be powers-of-two (e.g., $p = 2^8$ and $q = 2^{32}$) to take advantage of native machine arithmetic. This makes SimplePIR very suitable for GPU acceleration [50, 45, 44].

The main drawback of SimplePIR is the need for the client to download a large hint. Typically, when the database is arranged as a square (i.e., setting $\ell_1 = \ell_2 = \sqrt{N}$), this means the hint has size $n \cdot \sqrt{N}$. A typical setting takes $n \geq 2^{10}$, so the hint is concretely large (e.g., over 100 MB) even for relatively small databases (e.g., 1 GB).

Removing the hint using RLWE. To address this limitation, a number of works [39, 51, 63, 59] have introduced new variants of SimplePIR to compress the hint. Consider the matrix obtained by stacking the SimplePIR hint with the response: $\mathbf{C} = \begin{bmatrix} \mathbf{H} \\ \mathbf{r} \end{bmatrix} \in \mathbb{Z}_q^{(n+1) \times \ell_2}$. The columns of $\mathbf{C}$ are LWE encryptions of the elements in row i of the database, and the decoding algorithm is LWE decryption. Here, $\mathbf{C}$ is large because LWE ciphertexts have poor rate: encrypting a single scalar (over $\mathbb{Z}_p$) requires an $(n+1)$ -dimensional vector (over $\mathbb{Z}_q$). In this case, the ciphertext expansion factor is $(n+1) \log q / \log p$. The strategy taken in the aforementioned works is to switch the LWE ciphertexts to RLWE ciphertexts which have a much smaller expansion factor of $2 \log q / \log p$. The works of Tiptoe [39] and HintlessPIR [51] take a bootstrapping-based approach where they encrypt the LWE decryption key $\mathbf{s}$ under RLWE and homomorphically decrypt the ciphertext under RLWE. The works of YPIR [63] and InsPIRe [59] take a packing-based approach where they pack the ℓ_2 LWE ciphertexts into ℓ_2/n RLWE ciphertexts. The YPIR protocol uses the packing algorithm of Chen et al. [19], while the InsPIRe protocol develops a more refined packing algorithm (Fig. 1). We compare the two ring packing approaches in Remark 3.1. There, we argue that the InsPIRe approach is significantly more amenable to GPU acceleration.

InsPIRe ring packing. In this work, we follow the ring packing approach from InsPIRe [59] and show how to accelerate their approach using GPUs. We start by recalling their approach. Formally, the packing algorithm takes as input n LWE ciphertexts $(\mathbf{a}_0, b_0), \dots, (\mathbf{a}_{n-1}, b_{n-1})$ encrypted under a secret key $\mathbf{s} \in \mathbb{Z}_q^n$, where for all $k \in [0, n-1]$, we have $b_k = -\mathbf{s}^\top \mathbf{a}_k + \Delta m_k + e_k$. Define the polynomial $s = \sum_{i \in [0, n-1]} s_i x^i \in R_q$. The output of the packing algorithm is a single RLWE ciphertext $(a^*, b^*) \in R_q^2$ encrypting the polynomial $\sum_{i \in [0, n-1]} m_i x^i$ under s . The InsPIRe ring packing algorithm operates as follows:

- **Lift to the polynomial ring.** For each $k \in [0, n-1]$, write $\mathbf{a}_k = (a_{k,0}, \dots, a_{k,n-1})$ and let $\tilde{a}_k = \sum_{i \in [0, n-1]} a_{k,i} x^{-i} \in$

R_q . Then the constant coefficient of $s\tilde{a}_k$ is precisely $\sum_{i \in [0, n-1]} s_i a_{k,i} = \mathbf{s}^\top \mathbf{a}_k$. This means

$$b_k + s\tilde{a}_k = \Delta m_k + e_k + r_k \in R_q,$$

where $r_k \in R_q$ is a polynomial with constant coefficient 0.

- **Zero out non-constant terms via the trace.** Over $R = \mathbb{Z}[x]/(x^n + 1)$ where $n > 4$ is a power-of-two, the trace operator $\text{Tr}: R \rightarrow R$ can be written as

$$\text{Tr}(r) = \sum_{j=0}^{n/2-1} (\tau_5^j(r) + \tau_{2n-1}(\tau_5^j(r))),$$

where τ_ℓ is the ring automorphism $r(x) \mapsto r(x^\ell)$. The trace operator has the property that for all polynomials $r(x) = \sum_{i \in [0, n-1]} r_i x^i$, $\text{Tr}(r) = nr_0$. Namely, the trace projects a polynomial onto its constant coefficient scaled by the ring dimension. The server can homomorphically apply the trace operator to $(\tilde{a}_k, b_k)$ to obtain an encryption of $n \cdot m_k$.

- **Combining into a single ciphertext.** Given encryptions of $n \cdot m_0, \dots, n \cdot m_{n-1}$, the server can homomorphically combine them to an RLWE encryption (a^*, b^*) of the polynomial $\sum_{i \in [0, n-1]} n \cdot m_i x^i$. Specifically, the server scales the i^{th} ciphertext by the monomial x^i and sums the ciphertexts. This yields an RLWE encryption of the *packed* response. An easy way to eliminate the scaling factor is to scale the initial LWE ciphertexts $(\mathbf{a}_k, b_k)$ by $n^{-1} \bmod q$ prior to packing. This is well-defined as long as $\gcd(n, q) = 1$.

We give the formal description of this algorithm in Fig. 1 and state the correctness property below.

Theorem 2.2 (InsPIRe Packing [59, Theorem 2, adapted]). *Let n be a power of two, q be a modulus with $\gcd(n, q) = 1$, and $R = \mathbb{Z}[x]/(x^n + 1)$. Let $z \in \mathbb{N}$ be a decomposition base and $t = \lceil \log_z q \rceil$. Let $(\mathbf{a}_0, b_0), \dots, (\mathbf{a}_{n-1}, b_{n-1}) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ be a collection of LWE ciphertexts encrypting messages $m_0, \dots, m_{n-1}$ with secret $\mathbf{s} \in \mathbb{Z}_q^n$, where each error is subgaussian with parameter σ_{in} . Suppose $(\mathbf{a}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_5}), (\mathbf{a}_{\text{ks}, \tau_{2n-1}}, \mathbf{b}_{\text{ks}, \tau_{2n-1}}) \in R_q^t \times R_q^t$ are key-switching parameters associated with the automorphisms τ_5 and τ_{2n-1} , respectively, with secret key $s = \sum_{i \in [0, n-1]} s_i x^i \in R_q$, decomposition base z , and errors sampled from a discrete Gaussian distribution with parameter σ_{ks} . Applied to these inputs, and under the independence heuristic, the packing algorithm from Fig. 1 outputs an RLWE ciphertext $(a^*, b^*) \in R_q^2$ encrypting the polynomial $\sum_{i \in [0, n-1]} m_i x^i$ with respect to s and error e_{out} , and where the coefficients of e_{out} are subgaussian with variance $\sigma_{\text{out}}^2 \leq \sigma_{\text{in}}^2 + tn^2 z^2 \sigma_{\text{ks}}^2 / 4$.*

3 SandwichPIR Design

In this section, we describe the design of SandwichPIR. The key observation we make in this work is that the bulk of the

Setup: Let $n \in \mathbb{N}$ be a power of two, $q \in \mathbb{N}$ be a modulus where $\gcd(n, q) = 1$, and $R = \mathbb{Z}[x]/(x^n + 1)$. Let $z \in \mathbb{N}$ be a decomposition base and let $t = \lceil \log_z q \rceil$. Let τ_ℓ be the automorphism $r(x) \mapsto r(x^\ell)$.

Inputs: LWE ciphertexts $(\mathbf{a}_0, b_0), \dots, (\mathbf{a}_{n-1}, b_{n-1}) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ and a pair of key-switching keys $(\mathbf{a}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_5}), (\mathbf{a}_{\text{ks}, \tau_{2n-1}}, \mathbf{b}_{\text{ks}, \tau_{2n-1}}) \in R_q^t \times R_q^t$ associated with the automorphisms τ_5 and τ_{2n-1} , respectively.

Algorithm description:

1. **Trace evaluation:** For each $k \in [0, n-1]$, compute $\tilde{a}_k = n^{-1} \sum_{i \in [0, n-1]} a_{k,i} x^{-i} \in R_q$. Then, define $\tilde{\mathbf{a}}'_k \in R_q^n$ where

$$\forall 0 \leq j < n/2 : \quad \tilde{a}'_{k,j} = \tau_5^j(\tilde{a}_k) \quad \text{and} \quad \tilde{a}'_{k, n/2+j} = \tau_{2n-1}(\tau_5^j(\tilde{a}_k)).$$

2. **Aggregation:** Let $\tilde{\mathbf{a}}_{\text{agg}} = \sum_{k \in [0, n-1]} \tilde{\mathbf{a}}'_k \cdot x^k \in R_q^n$ and $\tilde{b}_{\text{agg}} = \sum_{k \in [0, n-1]} b_k \cdot x^k \in R_q$.

3. **Key-switching:** Let $\beta_{n/2-1} = \tilde{b}_{\text{agg}}$. Then, proceed as follows:

- Let $\alpha_{n/2-1} = \tilde{a}_{\text{agg}, n/2-1}$. For $\ell = n/2 - 1, \dots, 1$, perform the following key-switches:

$$\begin{aligned} \beta_{\ell-1} &= \beta_\ell + \tau_5^{\ell-1}(\mathbf{b}_{\text{ks}, \tau_5}^\top) \cdot \mathbf{g}_z^{-1}(\alpha_\ell) \\ \alpha_{\ell-1} &= \tilde{a}_{\text{agg}, \ell-1} + \tau_5^{\ell-1}(\mathbf{a}_{\text{ks}, \tau_5}^\top) \cdot \mathbf{g}_z^{-1}(\alpha_\ell). \end{aligned} \tag{2.2}$$

- Let $\alpha_{n-1} = \tilde{a}_{\text{agg}, n-1}$ and $\beta_{n-1} = \beta_0$. For $\ell = n/2 - 1, \dots, 1$, perform the following key-switches:

$$\begin{aligned} \beta_{n/2+\ell-1} &\leftarrow \beta_{n/2+\ell} + \tau_{2n-1}(\tau_5^{\ell-1}(\mathbf{b}_{\text{ks}, \tau_5}^\top)) \cdot \mathbf{g}_z^{-1}(\alpha_{n/2+\ell}) \\ \alpha_{n/2+\ell-1} &\leftarrow \tilde{a}_{\text{agg}, n/2+\ell-1} + \tau_{2n-1}(\tau_5^{\ell-1}(\mathbf{a}_{\text{ks}, \tau_5}^\top)) \cdot \mathbf{g}_z^{-1}(\alpha_{n/2+\ell}). \end{aligned} \tag{2.3}$$

Compute and output (a^*, b^*) where $a^* = \alpha_0 + \mathbf{a}_{\text{ks}, \tau_{2n-1}}^\top \mathbf{g}_z^{-1}(\alpha_{n/2})$ and $b^* = \beta_{n/2} + \mathbf{b}_{\text{ks}, \tau_{2n-1}}^\top \mathbf{g}_z^{-1}(\alpha_{n/2})$.

Figure 1: The InsPIRe packing algorithm [59].

InsPIRe ring packing algorithm in Fig. 1 boils down to a dense matrix-vector multiplication between a vector derived from the client’s query and a fixed matrix that only depends on query-independent components. Since the SimplePIR pass is also a matrix-vector multiplication between the client’s vector and the database, this means that nearly all of the server computation reduces to two essentially independent matrix-vector multiplications. This makes the overall protocol very amenable to GPU acceleration. The only wrinkle is that the two matrix multiplications must happen over two different moduli: (1) a power-of-two modulus for the SimplePIR pass; and (2) an NTT-friendly² modulus for the ring packing. In SandwichPIR, we adapt classic modulus switching ideas from [14, 13, 50] to “sandwich” a power-of-two modulus between two NTT-friendly moduli that we use (one for the query and one for response packing). As we discuss below, our variant of modulus switching is critical to reducing the noise accumulation from homomorphic operations.

²When $R = \mathbb{Z}[x]/(x^n + 1)$ and n is a power-of-two, we say a modulus q is NTT-friendly if $q = 1 \bmod 2n$. In this case, the number-theoretic transform (NTT) gives an isomorphism between the rings R_q and $\mathbb{Z}_q^n$.

3.1 Ring Packing as a Matrix Multiplication

The authors of InsPIRe [59] observe that if the message-independent components of the input LWE ciphertexts $\mathbf{a}_0, \dots, \mathbf{a}_{n-1}$ as well as the corresponding components $\mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}}$ in the key-switching parameters are known in advance, then the message-independent component a^* of the output can be computed in advance (see Fig. 1). The only piece that needs to be computed in the online phase is the message-dependent component b^* . A priori, the interleaved sequence of automorphisms and key-switching operations in Fig. 1 might suggest that this step is sequential and less amenable to GPU acceleration, and indeed, this was the argument made in [44, §3.5.3]. As a result, earlier works either move the ciphertext packing back to the CPU [50] or design new packing protocols that are more amenable to parallelization [44]. Here, we show that by viewing the algebraic computation through the right lens, the InsPIRe ring packing algorithm is in fact computing a dense matrix-vector product between the client’s query vector and a fixed query-independent matrix (known to the server).

Ring packing in the NTT domain. The key insight in our ring packing procedure is that each update in Eqs. (2.2) and (2.3) consists of an inner product between an automorphism applied to a *single* key-switching vector $\mathbf{b}_{\text{ks}, \tau_5}$ (that depends on the query) and a *query-independent* vector $\mathbf{g}_z^{-1}(\alpha_i)$. If we view each ring element in its NTT representation (i.e., view each ring element as a vector over $\mathbb{Z}_q^n$ consisting of evaluations at the primitive roots of unity), then the automorphism is applying a permutation to the entries of each vector. Thus, under the NTT representation, each of these inner products is computing *some* linear function between the NTT components of $\mathbf{b}_{\text{ks}, \tau_5}$ and those of $\mathbf{g}_z^{-1}(\alpha_i)$. The entire update can thus be described as a linear function of the NTT representation of $\mathbf{b}_{\text{ks}, \tau_5}$ and that of the query-independent components $\mathbf{g}_z^{-1}(\alpha_0), \dots, \mathbf{g}_z^{-1}(\alpha_{n-1})$.

More formally, we start by unrolling the computation of the message-dependent component $b^* \in R_q$ in Fig. 1:

$$b^* = \tilde{b}_{\text{agg}} + \mathbf{b}_{\text{ks}, \tau_{2n-1}}^\top \mathbf{g}_z^{-1}(\alpha_{n/2}) + b_{\text{ks}, \tau_5}^* \quad (3.1)$$

where

$$\begin{aligned} b_{\text{ks}, \tau_5}^* &= \sum_{j \in [n/2-1]} (\tau_5^{j-1}(\mathbf{b}_{\text{ks}, \tau_5}))^\top \mathbf{g}_z^{-1}(\alpha_j) \\ &+ \sum_{j \in [n/2-1]} (\tau_{2n-1}(\tau_5^{j-1}(\mathbf{b}_{\text{ks}, \tau_5})))^\top \mathbf{g}_z^{-1}(\alpha_{n/2+j}) \end{aligned} \quad (3.2)$$

The highlighted terms in Eqs. (3.1) and (3.2) are all functions of $\alpha_0, \dots, \alpha_{n-1}$, which are functions of the message-independent components $\mathbf{a}_0, \dots, \mathbf{a}_{n-1}, \mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}}$. All of these values are known to the server in the offline phase.

- Suppose $q = 1 \bmod 2n$. Then, the number-theoretic transform NTT: $R_q \rightarrow \mathbb{Z}_q^n$ is an isomorphism between the ring R_q and the ring $\mathbb{Z}_q^n$, where we view NTT(r) as a *column* vector. We extend NTT to operate component-wise on vectors. Next, for all $j \in \mathbb{Z}_{2n}^*$, there is a permutation $\pi_j: [n] \rightarrow [n]$ where for all $r \in R_q$, and setting $r' = \tau_j(r)$, $\hat{r} = \text{NTT}(r)$, and $\hat{r}' = \text{NTT}(r')$, it holds that $\hat{r}'_i = \hat{r}_{\pi_j(i)}$ for all $i \in [n]$.
- Next, define the following quantities:

$$\begin{aligned} \hat{\beta} &= (\hat{\beta}_1, \dots, \hat{\beta}_n)^\top = \text{NTT}(b_{\text{ks}, \tau_5}^*) \in \mathbb{Z}_q^n \\ \forall j \in [0, n-1] : \hat{\mathbf{A}}_j &= \text{NTT}((\mathbf{g}_z^{-1}(\alpha_j))^\top) \in \mathbb{Z}_q^{n \times t} \\ \hat{\mathbf{B}} &= \text{NTT}(\mathbf{b}_{\text{ks}, \tau_5}^\top) \in \mathbb{Z}_q^{n \times t} \end{aligned}$$

Let $\hat{\mathbf{a}}_{j,i}^\top, \hat{\mathbf{b}}_i^\top \in \mathbb{Z}_q^t$ denote the i^{th} row of $\hat{\mathbf{A}}_j$ and $\hat{\mathbf{B}}$, respectively. For each $j \in [n/2-1]$, let $\hat{\pi}_j(i) := \pi_5^{j-1}(i)$ and $\hat{\pi}'_j(i) := \pi_{2n-1}(\pi_5^{j-1}(i))$. By Eqs. (3.1) and (3.2) and

grouping terms together, we have

$$\begin{aligned} \hat{\beta}_i &= \sum_{j \in [n/2-1]} \hat{\mathbf{b}}_{\hat{\pi}_j(i)}^\top \hat{\mathbf{a}}_{j,i} + \sum_{j \in [n/2-1]} \hat{\mathbf{b}}_{\hat{\pi}'_j(i)}^\top \hat{\mathbf{a}}_{n/2+j,i} \\ &= \sum_{k \in [n]} \hat{\mathbf{b}}_k^\top \cdot \underbrace{\left(\sum_{\substack{j \in [n/2-1]: \\ \hat{\pi}_j(i)=k}} \hat{\mathbf{a}}_{j,i} + \sum_{\substack{j \in [n/2-1]: \\ \hat{\pi}'_j(i)=k}} \hat{\mathbf{a}}_{n/2+j,i} \right)}_{\hat{\mathbf{u}}_{i,k}}. \end{aligned} \quad (3.3)$$

- From Eq. (3.3), we have that for each $i \in [n]$, $\hat{\beta}_i = \sum_{k \in [n]} \hat{\mathbf{b}}_k^\top \hat{\mathbf{u}}_{i,k}$. Define the vectors

$$\begin{aligned} \hat{\mathbf{b}}^\top &= [\hat{\mathbf{b}}_1^\top \mid \dots \mid \hat{\mathbf{b}}_n^\top] \in \mathbb{Z}_q^{nt} \\ \hat{\mathbf{u}}_i^\top &= [\hat{\mathbf{u}}_{i,1}^\top \mid \dots \mid \hat{\mathbf{u}}_{i,n}^\top] \in \mathbb{Z}_q^{nt}. \end{aligned}$$

Then, $\hat{\beta}_i = \hat{\mathbf{b}}^\top \hat{\mathbf{u}}_i$. If we now define $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1 \mid \dots \mid \hat{\mathbf{u}}_n] \in \mathbb{Z}_q^{nt \times n}$, then $\hat{\beta}^\top = \hat{\mathbf{b}}^\top \hat{\mathbf{U}} \in \mathbb{Z}_q^n$. Moreover, the matrix $\hat{\mathbf{U}}$ only depends on the message-independent components $\mathbf{a}_0, \dots, \mathbf{a}_{n-1}, \mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}}$ whereas the vector $\hat{\mathbf{b}}^\top$ is obtained by concatenating together the NTT representation of the query component $\mathbf{b}_{\text{ks}, \tau_5}$.

Taken together, computing b^* in Eq. (3.1) reduces to computing a single dense matrix-vector product between the vector $\hat{\mathbf{b}} \in \mathbb{Z}_q^{nt}$ derived from the client's query and the matrix $\hat{\mathbf{U}} \in \mathbb{Z}_q^{nt \times n}$ derived from the message-independent components. After computing b_{ks, τ_5}^* in Eq. (3.2), the only remaining computation is $\mathbf{b}_{\text{ks}, \tau_{2n-1}}^\top \mathbf{g}_z^{-1}(\alpha_{n/2})$. In NTT representation, this requires nt point-wise multiplications and a summation.

Remark 3.1 (Comparison with YPIR Packing). The reason the InsPIRe ring packing approach [59] is amenable to GPU acceleration is that the bulk of the computation in Fig. 1 is a linear function of a *single* key-switching key $\mathbf{b}_{\text{ks}, \tau_5}$. This allows us to unroll the ring packing into a single matrix-vector product between the NTT components of $\mathbf{b}_{\text{ks}, \tau_5}$ and query-independent values. In contrast, the Chen et al. [19] ring packing procedure used in YPIR [63] takes a tree-based approach where each level of the tree applies a different automorphism ($\log n$ in total), so the same optimization does not apply. Moreover, as shown in [59], the procedure in Fig. 1 introduces less noise. This is the reason we focus exclusively on the InsPIRe packing procedure in this work.

3.2 The SandwichPIR Protocol

SandwichPIR follows the same template as YPIR [63] and the base version of InsPIRe [59] (i.e., composing SimplePIR with ring packing). The full InsPIRe protocol also includes a homomorphic polynomial evaluation technique to further reduce communication, which we do not consider in this work. We focus on accelerating SimplePIR together with the InsPIRe ring packing algorithm. We describe this template below:

- **Setup:** The public parameters define a public matrix $A \in \mathbb{Z}_q^{n \times \ell_1}$ (used to encrypt the SimplePIR query) and the key-switching components a_{ks, τ_5} and $a_{ks, \tau_{2n-1}}$ (for ring packing). In the setup phase, the server takes the database $D \in \mathbb{Z}_p^{\ell_1 \times \ell_2}$ and computes the SimplePIR hint $H = AD$. The columns of H correspond to the message-independent components of the LWE ciphertext that are used for ring packing. Using H , a_{ks, τ_5} , and $a_{ks, \tau_{2n-1}}$, the server computes the fixed matrices $\hat{U}_1, \dots, \hat{U}_K$ that will be used for ring packing, where $K = \ell_2/n$ is the number of blocks (i.e., ring packing packs ℓ_2 LWE ciphertexts into K RLWE ciphertexts).
- **Client’s query:** To query for a record in row i , the client encrypts the i^{th} indicator vector using LWE to obtain an encrypted query $q \in \mathbb{Z}_q^{\ell_1}$. In addition, the client prepares the message-dependent components b_{ks, τ_5} and $b_{ks, \tau_{2n-1}}$ of the key-switching keys. The query consists of q , b_{ks, τ_5} , $b_{ks, \tau_{2n-1}}$.
- **Server response:** The server computes the SimplePIR response $b^T = q^T D$. Then, the server applies ring packing to the LWE ciphertexts $C = \begin{bmatrix} H \\ b^T \end{bmatrix} \in \mathbb{Z}_q^{(n+1) \times \ell_2}$. Specifically, the server groups C into ℓ_2/n groups of n columns each and evaluates the packing algorithm from Fig. 1. Using the procedure in Section 3.1, the bulk of this computation reduces to matrix-vector products between the NTT components of b_{ks, τ_5} and the precomputed matrices $\hat{U}_1, \dots, \hat{U}_K$.
- **Answer decoding:** To recover the message, the client decrypts the relevant ciphertext that contains their requested record and reads off the relevant coefficient.

One issue with the above template is that each step favors a different type of ciphertext modulus q :

- **Preprocessing prefers an NTT-friendly modulus.** Computing $H = AD$ takes $\Theta(\ell_1 \ell_2 n)$ operations. As shown in [63, 50], we can reduce this to $\Theta(\ell_1 \ell_2 \log n)$ if A is negacyclic. Choosing A this way also reduces the client’s encryption cost. Using NTTs forces us to set $q = 1 \bmod 2n$.
- **The linear scan prefers a power-of-two modulus.** With $q = 2^{32}$ and the database consisting of elements over $\mathbb{Z}_p$ with $p = 2^8$, the matrix product $q^T D$ runs in native 32-bit integer arithmetic. On tensor cores, the accumulator performs the modular reduction for free. The alternative of running the scan over R_q throughout forces the server to store the database in NTT representation over $\mathbb{Z}_q$. This increases the in-memory size by $\log q / \log p$ and thus, reduces the throughput by the same factor [63, Remark 4.1].
- **Packing requires an NTT-friendly modulus.** Our approach of recasting ring packing as a matrix multiplication in Section 3.1 relies on the NTT which again necessitates choosing $q = 1 \bmod 2n$.

SandwichPIR satisfies all three requirements by using an NTT-friendly modulus q_{out} for preprocessing and packing and a power-of-two modulus q_{in} for the linear scan (i.e., the power-of-two modulus is “sandwiched” between the NTT-

friendly modulus). Between each step, SandwichPIR applies a variant of modulus switching [14, 13], where we only scale the message-dependent component of the ciphertext. As we show in Appendix A.1, this is correct because the initial and final moduli are the same. We also show that if we were to apply standard modulus switching (which scales both the message-dependent and the message-independent components of the ciphertext), then our scheme would exceed its noise budget and require picking substantially larger parameters. In contrast, our variant of modulus switching introduces very small additional noise. At the very end of the protocol, we apply one final modulus switching to reduce the ciphertext size; this step uses the standard split modulus switching from [62]. We give the full description of SandwichPIR in Fig. 2 and the correctness analysis in Appendix A.1. Security of the protocol relies on standard (circular-secure) ring LWE; this is the same as in InsPIRe since the query structure in our scheme is the same as that of InsPIRe. We provide more details in Appendix A.2.

Cross-client batching. Cross-client batching [56, 63, 50] is a simple way to improve the effective throughput of lattice-based PIR. The goal is to use a single linear scan over the database to answer queries from multiple independent clients. In SandwichPIR, the server processing essentially boils down to a collection of matrix-vector products between a vector corresponding to the client’s query and a matrix from the encoded database. This is the bulk of the computation followed by some lightweight post-processing. To process queries from multiple independent clients, the server can simply stack the queries together so each matrix-vector multiplication transforms into a matrix-matrix multiplication. This makes the protocol particularly well-suited for GPU acceleration.

Remark 3.2 (SandwichPIR with an Offline Hint). While SandwichPIR is designed for the silent preprocessing model where there is no offline communication, we note that we can reduce the online communication if we allow for a small offline hint. Namely, the message-independent components $a'_1, \dots, a'_K$ of the response ciphertexts in Fig. 2 are query-independent and could be communicated in a separate offline phase. Essentially, we can view these components as the “compressed” SimplePIR hint. For settings where a client makes multiple queries to the database, these components need only to be communicated once and can subsequently be reused. We note that this optimization can also be applied to many other hintless schemes (e.g., [51, 63, 59]).

4 SandwichPIR Implementation

We design SandwichPIR to take advantage of modern Nvidia GPUs. Modern GPUs organize computation around streaming multiprocessors, each of which executes many

Parameters: Let n be a power of two, $q_{\text{out}} = 1 \bmod 2n$, and q_{in} be a power of two with $q_{\text{out}} < q_{\text{in}}$. Let $R_{q_{\text{out}}} = \mathbb{Z}_{q_{\text{out}}}[x]/(x^n + 1)$. Let p be a plaintext modulus and $q_{\text{fin},1} \geq q_{\text{fin},2}$ be output moduli. Let σ be a Gaussian width parameter, z be the gadget base, and $t = \lceil \log_z q_{\text{out}} \rceil$.

Algorithms:

- **Setup**($1^\lambda, \mathbf{D}$): On input the security parameter λ and the database $\mathbf{D} \in \mathbb{Z}_p^{\ell_1 \times \ell_2}$:
 1. Sample $\mathbf{A} \xleftarrow{R} \mathbb{Z}_{q_{\text{out}}}^{n \times \ell_1}$ and $\mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}} \xleftarrow{R} R_{q_{\text{out}}}^t$. In the implementation, we set $\mathbf{A}$ to be a random nega-cyclic matrix (as in [63, 50]).
 2. Compute $\mathbf{H} = \mathbf{A}\mathbf{D} \in \mathbb{Z}_{q_{\text{out}}}^{n \times \ell_2}$. Parse $\mathbf{H} = [\mathbf{H}_1 \mid \dots \mid \mathbf{H}_K]$ where $\mathbf{H}_k \in \mathbb{Z}_{q_{\text{out}}}^{n \times n}$.
 3. For each $k \in [K]$, compute $\hat{\mathbf{U}}_k$ from $\mathbf{H}_k, \mathbf{a}_{\text{ks}, \tau_5}$, and $\mathbf{a}_{\text{ks}, \tau_{2n-1}}$ as described in Section 3.1. Here, the columns of $\mathbf{H}_k$ are the message-independent components of the LWE ciphertext.
 4. Output $\text{pp} = (\mathbf{A}, \mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}})$ and $\text{edb} = (\mathbf{D}, \mathbf{H}, \hat{\mathbf{U}}_1, \dots, \hat{\mathbf{U}}_K)$.
- **Query**($\text{pp}, (i_1, i_2)$): On input the public parameters $\text{pp} = (\mathbf{A}, \mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}})$ and the index (i_1, i_2) :
 1. Sample $s \leftarrow D_{R, \sigma}$ and let $\mathbf{s} \in \mathbb{Z}^n$ be the coefficient vector.
 2. Sample $\mathbf{e} \leftarrow D_{\mathbb{Z}, \sigma}^{\ell_1}$ and compute $\mathbf{q}_0^\top = -\mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top + \lfloor q_{\text{out}}/p \rfloor \mathbf{u}_{i_1}^\top \in \mathbb{Z}_{q_{\text{out}}}^{\ell_1}$ where $\mathbf{u}_{i_1}$ is the indicator vector for row i_1 . Let $\mathbf{q} = \lfloor (q_{\text{in}}/q_{\text{out}}) \cdot \mathbf{q}_0 \rfloor \in \mathbb{Z}_{q_{\text{in}}}^{\ell_1}$.
 3. For each $\tau \in \{\tau_5, \tau_{2n-1}\}$, sample $\mathbf{e}_{\text{ks}, \tau} \leftarrow D_{R, \sigma}^t$ and compute $\mathbf{b}_{\text{ks}, \tau}^\top = -s \cdot \mathbf{a}_{\text{ks}, \tau}^\top + \mathbf{e}_{\text{ks}, \tau}^\top + \tau(s) \cdot \mathbf{g}_z^\top \in R_{q_{\text{out}}}^t$.
 4. Output $\text{st} = (s, i_2)$ and $\text{qu} = (\mathbf{q}, \mathbf{b}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_{2n-1}})$.
- **Answer**(edb, qu): On input the encoded database $\text{edb} = (\mathbf{D}, \mathbf{H}, \hat{\mathbf{U}}_1, \dots, \hat{\mathbf{U}}_K)$ and the query $\text{qu} = (\mathbf{q}, \mathbf{b}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_{2n-1}})$:
 1. **Linear scan.** Compute $\mathbf{r}_0^\top = \mathbf{q}^\top \mathbf{D} \in \mathbb{Z}_{q_{\text{in}}}^{\ell_2}$ and set $\mathbf{r}^\top = \lfloor (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{r}_0^\top \rfloor \in \mathbb{Z}_{q_{\text{out}}}^{\ell_2}$.
 2. **Ring packing.** Let $\mathbf{C} = \begin{bmatrix} \mathbf{H} \\ \mathbf{r}^\top \end{bmatrix} \in \mathbb{Z}_{q_{\text{out}}}^{(n+1) \times \ell_2}$ and parse $\mathbf{C} = [\mathbf{C}_1 \mid \dots \mid \mathbf{C}_K]$ where $\mathbf{C}_k \in \mathbb{Z}_{q_{\text{out}}}^{(n+1) \times n}$. For each $k \in [K]$, apply the packing algorithm of Fig. 1 to $\mathbf{C}_k$ (using $\mathbf{b}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_{2n-1}}, \hat{\mathbf{U}}_k$ as described in Section 3.1) to obtain $(a_k, b_k) \in R_{q_{\text{out}}}^2$.
 3. **Modulus switching.** For each $k \in [K]$, compute $a'_k = \lfloor (q_{\text{fin},1}/q_{\text{out}}) \cdot a_k \rfloor \in R_{q_{\text{fin},1}}$ and $b'_k = \lfloor (q_{\text{fin},2}/q_{\text{out}}) \cdot b_k \rfloor \in R_{q_{\text{fin},2}}$. Output $\text{ans} = ((a'_1, b'_1), \dots, (a'_K, b'_K))$.
- **Recover**(st, ans): On input $\text{st} = (s, i_2)$ and $\text{ans} = ((a'_1, b'_1), \dots, (a'_K, b'_K))$, write $i_2 = n(k-1) + j$ where $k \in [K]$ and $j \in [n]$, compute $v' = \lfloor (q_{\text{fin},2}/q_{\text{fin},1}) \cdot (-sa'_k) \rfloor + b'_k \in R_{q_{\text{fin},2}}$ and $v = \lfloor (p/q_{\text{fin},2}) \cdot v' \rfloor \in R_p$. Output the coefficient of x^{j-1} in v .

Figure 2: The SandwichPIR protocol.

groups of threads (e.g., of size 32) that advance in lockstep. Every instruction is issued to a group as a whole, and each thread in the group executes it on its own data. A streaming multiprocessor contains two main types of arithmetic units:

- The first are general-purpose cores, which perform ordinary scalar arithmetic. Each instruction produces a single scalar output per thread. In this work, we use these cores to perform 32-bit integer operations (int32).
- The second are special-purpose high-throughput tensor cores, which implement matrix-multiply-accumulate instructions. Here, a *single* instruction issued to a thread group takes as input a triple of matrices ($\mathbf{A}, \mathbf{B}, \mathbf{C}$) of fixed, hardware-defined dimensions, and outputs $\mathbf{AB} + \mathbf{C}$. A single instruction thus performs an entire matrix product rather than a single multiply-accumulate per thread.

Tensor cores are optimized for settings where the input matrices $\mathbf{A}$ and $\mathbf{B}$ have lower precision than the accumulator. In this work, we consider the case where $\mathbf{A}$ and $\mathbf{B}$ are matrices of 8-bit integers (int8) while the accumulator $\mathbf{C}$ and the output $\mathbf{AB} + \mathbf{C}$ consist of 32-bit integers (int32).

Tensor cores substantially increase the computational throughput on Nvidia GPUs. The Nvidia L40S that we use for evaluation is rated at 733 trillion int8 operations per second (TOPS) on its tensor cores compared to roughly 45 TOPS of int32 arithmetic on its general-purpose cores. A typical server-class CPU (specifically, a 4-core Intel Sapphire Rapids at 3.2 GHz) achieves 3.3 TOPS with AVX-512 VNNI. This is over 200× lower than that of the L40S. Looking at memory bandwidth, the L40S provides 805 GB/s whereas the CPU provides roughly 53 GB/s. We refer to Table 4 for a comparison.

Implementation details. We implement SandwichPIR in $\sim 2,700$ lines of CUDA and $\sim 7,500$ lines of Rust. We use the CUTLASS [75] library for the main matrix multiplications (for the server computation in Fig. 2) and write custom kernels for the remaining server operations. We summarize our main design considerations below:

- **The linear scan.** The linear scan in the Answer algorithm (i.e., computing $\mathbf{q}^T \mathbf{D}$; see Fig. 2) is the most computationally demanding step of the server computation. As described in Section 3.2, we take the plaintext modulus to be $p = 2^8$ and the ciphertext modulus for the linear scan to be a power-of-two: namely, $q_{\text{in}} = 2^{32}$. This directly lines up with evaluation on tensor cores since the database entries consist of 8-bit inputs and the output is taken modulo 2^{32} . The only issue is that the query vector $\mathbf{q}$ is over $\mathbb{Z}_{2^{32}}$. To evaluate this using tensor cores, we first decompose $\mathbf{q} = \mathbf{q}_0 + 2^8 \cdot \mathbf{q}_1 + 2^{16} \cdot \mathbf{q}_2 + 2^{24} \cdot \mathbf{q}_3$ where $\mathbf{q}_i \in \mathbb{Z}_{2^8}^{\ell_1}$. Let $\mathbf{Q} \in \mathbb{Z}_{2^8}^{4 \times \ell_1}$ be the matrix whose rows are $\mathbf{q}_0^T, \mathbf{q}_1^T, \mathbf{q}_2^T, \mathbf{q}_3^T$. The server first computes $\mathbf{QD}$ using tensor cores. Then, it combines the terms together using a second kernel that performs pointwise bit-shifts and accumulation.
- **Ring packing.** The ring packing needs to operate over an NTT-friendly modulus $q_{\text{out}} < 2^{32}$, so neither the input nor the output of the matrix multiplication are natively compatible with the tensor core abstraction. However, we can use a similar decomposition as before to implement the matrix multiplication. Recall from Section 3.1 that the ring packing essentially boils down to computing $\hat{\mathbf{b}}^T \hat{\mathbf{U}}'$ where $\hat{\mathbf{U}}' = [\hat{\mathbf{U}}_1 \mid \dots \mid \hat{\mathbf{U}}_K] \in \mathbb{Z}_{q_{\text{out}}}^{nt \times \ell_2}$ and $\hat{\mathbf{b}} \in \mathbb{Z}_{q_{\text{out}}}^{nt}$ is a vector derived from the client's query. In this case, we decompose

$$\begin{aligned}\hat{\mathbf{b}} &= \hat{\mathbf{b}}_0 + 2^8 \cdot \hat{\mathbf{b}}_1 + 2^{16} \cdot \hat{\mathbf{b}}_2 + 2^{24} \cdot \hat{\mathbf{b}}_3 \\ \hat{\mathbf{U}}' &= \hat{\mathbf{U}}'_0 + 2^8 \cdot \hat{\mathbf{U}}'_1 + 2^{16} \cdot \hat{\mathbf{U}}'_2 + 2^{24} \cdot \hat{\mathbf{U}}'_3.\end{aligned}$$

We compute each of the 16 pairwise products $\hat{\mathbf{b}}_i^T \hat{\mathbf{U}}'_j$ using tensor cores, and then recombine the pairwise products into $\hat{\mathbf{b}}^T \hat{\mathbf{U}}'$ using a post-processing kernel. Each pairwise product accumulates values of magnitude at most ntp^2 , which is smaller than q_{in} for our parameters. The intermediate values thus never overflow, and we only need to apply a single (lazy) modular reduction at the very end of the accumulation (rather than one per product). In our implementation, we run the linear scan and the ring packing sequentially since both saturate the tensor cores (with cross-client batching). We start with packing since it requires copying less data from CPU to GPU. With our parameters, the vector $\hat{\mathbf{b}}$ for ring packing is 32 KB while the SimplePIR query vector is typically over 500 KB. This way, copying the SimplePIR query from the CPU to the GPU can occur concurrently with the matrix multiplication for ring packing. We refer to Fig. 4 for microbenchmarks.

Table 1: Lattice Parameters for SandwichPIR.

Symbol	Description	Value
n	Ring dimension	2048
q_{out}	Outer packing modulus	$2^{32} - 12287$
q_{in}	Inner scan modulus	2^{32}
p	Plaintext modulus	256
σ	Gaussian width	$\sqrt{\pi}/2$
(z, t)	Decomposition base/length	$(2^8, 4)$
$(q_{\text{fin},1}, q_{\text{fin},2})$	Output modulus	$(2^{18}, 2^{10})$

Note: our choice of $\sigma = \sqrt{\pi}/2$ corresponds to a discrete Gaussian with standard deviation $s = \sigma/\sqrt{2\pi} = 0.5$.

- **Hint precomputation.** Similar to [63, 50], we sample $\mathbf{A}$ to be a random nega-cyclic matrix. Computing the hint $\mathbf{H} = \mathbf{AD}$ during Setup can then be done using NTTs with $O(\ell_1 \ell_2 \log n)$ operations rather than the naïve algorithm which requires $O(\ell_1 \ell_2 n)$ operations. We implement the classic radix-2 Cooley–Tukey algorithm [27] in CUDA. As above, we defer modular reductions to only when there is a possibility of overflow.
- **Packing precomputation.** During Setup, the server computes the matrices $\hat{\mathbf{U}}_1, \dots, \hat{\mathbf{U}}_K$ from the hint $\mathbf{H}$ and the key-switching components $\mathbf{a}_{\text{ks}, \tau_5}$ and $\mathbf{a}_{\text{ks}, \tau_{2n-1}}$. Following the procedure from Section 3.1, this requires computing gadget decompositions of polynomials defined by $\mathbf{H}$. This means converting between the coefficient and NTT representations of the underlying polynomials. For this, we reuse the same NTT kernels from hint precomputation. Like previous works [40, 63, 50, 59], we derive the public parameters $\mathbf{A}$, $\mathbf{a}_{\text{ks}, \tau_5}$, and $\mathbf{a}_{\text{ks}, \tau_{2n-1}}$ from a short seed using a PRF (ChaCha20 in our implementation). This way, the client and the server can each locally derive the public parameters from the seed without any communication.

Our implementation is open-source and available at <https://github.com/sidsabh/sandwichpir>.

Parameter selection. We instantiate the different parameters in Fig. 2 to achieve a correctness error of at most $\delta \leq 2^{-40}$ and to provide at least 128 bits of (classical) security, as determined by the lattice estimator [4] (commit 53da598). We summarize our parameter selection methodology as follows:

- We arrange the database as a matrix $\mathbb{Z}_p^{\ell_1 \times \ell_2}$. On each query, the client retrieves one row of the matrix (i.e., an element of $\mathbb{Z}_p^{\ell_2}$). We take ℓ_1 and ℓ_2 to be multiples of n (powers of two in our experiments). The client encrypts its query as ℓ_1/n RLWE ciphertexts, and the response contains $K = \ell_2/n$ packed RLWE ciphertexts. The query additionally includes key-switching keys for ring packing. Typically, for a database with N bytes, we set $\ell_1, \ell_2 \approx \sqrt{N}$. This roughly balances the communication and, in our experiments,

maximizes the server throughput (see Table 7).

- To best leverage tensor cores for the linear scan, we set $p = 2^8$ and the inner modulus to $q_{\text{in}} = 2^{32}$.
- We choose the largest NTT-friendly modulus q_{out} where $q_{\text{out}} < q_{\text{in}}$. This reduces the error from modulus switching (see Appendix A.1). For our specific parameters, the modulus switching contributes less than 1 bit to the overall noise.
- Finally, we pick the smallest ring dimension n and width parameter σ so that the associated RLWE instance (with parameters $n, q_{\text{out}}, \sigma$) provides at least 128 bits of security.
- We choose the largest gadget base z and the smallest modulus-switching targets $(q_{\text{fin},1}, q_{\text{fin},2})$ that ensure a correctness error at most $\delta < 2^{-40}$.

We summarize our parameter choices in Table 1.

5 Experimental Evaluation

We measure the performance of SandwichPIR on several AWS instance types. For our single-GPU experiments, we use an AWS g6e.xlarge instance, which has a single Nvidia L40S GPU with CUDA 12.4, 48 GB of device memory, and 32 GB of DRAM. For multi-GPU experiments, we use an AWS g6e.48xlarge instance, which has 8 Nvidia L40S, 384 GB of GPU memory, and 1536 GB DRAM. We benchmark all CPU baselines on an AWS r7i.2xlarge instance with 8 vCPUs (Intel Sapphire Rapids at 3.2 GHz) and 64 GB DRAM. Note that the CPU instance has 64 GB DRAM, which is more than the 48 GB device memory available to the GPU. We report all measurements based on an average over 10 trials. Throughout, we observe a relative standard deviation below 3%. We write KB, MB, GB, TB for 2^{10} , 2^{20} , 2^{30} , and 2^{40} bytes, respectively.

Cost model. When comparing schemes designed for different hardware architectures, we also report the monetary cost needed to support a million queries based on the current AWS cost model. Throughout, we use the AWS on-demand pricing. At the time of writing, the per-hour instance costs are: \$0.53 for r7i.2xlarge, \$1.86 for g6e.xlarge, and \$30.13 for g6e.48xlarge. Outgoing bandwidth is priced at \$0.09/10⁹ bytes and incoming bandwidth is free.

Benchmarking setup. We compare with the following schemes:

- The CPU-based schemes SimplePIR [40], HintlessPIR [51], YPIR [63], KsPIR [57], and InsPIRe [59]. All schemes except SimplePIR support silent preprocessing.
- The GPU-based implementation of SimplePIR [40] from [50]. This implementation uses int32 kernels to implement SimplePIR, which has lower throughput compared to the int8-int32 tensor cores used in our implementation.
- The GPU-based scheme PIRonGPU [71, 70] which combines GPU acceleration for homomorphic encryption [70] with SealPIR [6]. This scheme requires the client to upload a 2.9 MB client-specific state. While there have been subsequent GPU-based PIR schemes [78, 42, 45], their implementations are not publicly available at the time of writing.

We use the provided reference implementation of each underlying scheme. For SimplePIR, we use the implementation from [50], which supports both CPU and GPU. We fix an 8-bit message space and apply the same modulus switching used in SandwichPIR for response compression. Modulus switching reduces their per-query response to 40 KB and the hint to 144 MB, and is strictly favorable to SimplePIR in our comparisons. Since schemes in the SimplePIR family fetch a complete row of the database on each query, we focus on the large-record setting throughout this work.

Macrobenchmarks. In Table 2, we report the communication and computation costs needed to retrieve a 32 KB record from a 4 GB database. In Appendix B, we give the breakdown for 1 GB and 16 GB databases (see Table 8). On a 4 GB database, SandwichPIR answers a single query in 8 ms and achieves a server throughput of 488 GB/s. This is 88× the throughput of InsPIRe, the fastest CPU scheme without offline communication, and 57× faster than SimplePIR on the CPU. In terms of compute costs, SandwichPIR is 25× cheaper than InsPIRe and 16× cheaper than the CPU-based SimplePIR.

Our CPU baselines only consider a single-threaded execution, and we can potentially obtain a 4× improvement in throughput by running independent instances on each core. The r7i.2xlarge instances we use have 4 physical cores. Even with a 4× speed-up to each of the CPU schemes, the compute cost of SandwichPIR is still 6.2× lower than that of InsPIRe and 4.1× lower than that of the CPU-based SimplePIR. If we consider total online cost under these conditions, SandwichPIR remains 2.4× lower than that of InsPIRe (\$14.56 vs. \$35.24 for a million queries) and 1.4× lower than that of SimplePIR. If we allow the client to download a short hint (Remark 3.2), the online cost of SandwichPIR drops to \$7.93 for a million queries. This is 2.6× lower than the online cost of SimplePIR and we have simultaneously reduced the size of the offline hint by 2,048× (from 144 MB to 72 KB).

Compared to the GPU implementation of SimplePIR from [50], SandwichPIR achieves a 4.0× improvement in server throughput (488 GB/s vs. 123 GB/s). This difference is entirely due to how we implement arithmetic on the GPUs. The implementation of [50] performs the linear scan using int32 kernels on the CUDA cores, whereas we decompose the scan into int8 products that run on the tensor cores (Section 3.1). Note that we achieve this while *additionally* performing hint compression on the GPU, which the implementation from [50] does not. The combination of

Table 2: Performance comparison of different PIR schemes on a 4 GB database with 32 KB records. For each scheme, we report the offline communication (if any), the online upload (On $\uparrow$), the online download (On $\downarrow$), the server response time, and the server throughput (ratio of database size to response time). We report the server computation and communication costs based on the AWS cost model for handling a million queries. We also report the offline bandwidth cost needed to support a million clients. We refer to Table 8 in Appendix B for the same comparison at 1, 4, and 16 GB.

	Scheme	Communication			Server Time		Online Cost (per 10^6 queries)			Offline
		Off ↓	On ↑	On ↓	Time	Throughput	Compute	Bandwidth	Total	Cost
CPU	SimplePIR [40, 50]	144 MB	512 KB	40 KB	468 ms	8.55 GB/s	\$68.75	\$3.69	\$72.44	\$13,590
	HintlessPIR [51]	—	722 KB	2.97 MB	1,427 ms	2.80 GB/s	\$209.77	\$280.12	\$489.89	—
	YPIR [63]	—	1.33 MB	120 KB	925 ms	4.32 GB/s	\$135.98	\$11.06	\$147.04	—
	KsPIR [57]	—	2.53 MB	224 KB	2,745 ms	1.46 GB/s	\$403.45	\$20.64	\$424.09	—
	InsPIRe [59]	—	1.04 MB	96 KB	718 ms	5.57 GB/s	\$105.54	\$8.85	\$114.39	—
GPU	PIRonGPU [71]	—	144 KB	2.25 MB	3,075 ms	1.30 GB/s	\$1,589.60	\$212.34	\$1,801.94	—
	SimplePIR [40, 50] (int32)	144 MB	512 KB	40 KB	32.5 ms	123 GB/s	\$16.80	\$3.69	\$20.49	\$13,590
	SandwichPIR	—	576 KB	112 KB	8.19 ms	488 GB/s	\$4.24	\$10.32	\$14.56	—
	SandwichPIR (with hint)	72 KB	576 KB	40 KB	8.19 ms	488 GB/s	\$4.24	\$3.69	\$7.93	\$6.64

using tensor cores for compute and our adaptation of ring packing (i.e., hint compression) as a matrix multiplication (Section 3.1) allows us to perform *both* steps on the GPU at a higher throughput than previously known. As we show in Fig. 3, the gap widens even more when we consider batching.

Compared to PIRongPU [71], SandwichPIR achieves a 375 $\times$ improvement in server throughput (and shorter responses in exchange for longer queries). Both schemes retrieve a 32 KB record here, which PIRongPU obtains via four queries to a database with 8 KB records. The higher throughput in SandwichPIR is due to its design: the core server computation consists of two matrix products, which are directly amenable to GPU acceleration. In contrast, PIRongPU [71], as well as subsequent works like ShiftPIR [78] and GPIR [42], has focused on GPU acceleration of more complex homomorphic encryption machinery (e.g., leveraging homomorphic multiplication for response compression). The increased complexity introduces new challenges for acceleration. More recent works have nonetheless shown impressive results along these lines. For instance, GPIR obtains a 46 $\times$ improvement in throughput over PIRongPU and a 19 $\times$ improvement over ShiftPIR on a 4 GB database [42, Table 1], measured on an Nvidia RTX 5090 at a smaller 16 KB record size. SandwichPIR uses minimal machinery from homomorphic encryption and achieves a more substantial improvement over PIRongPU in our benchmarks.

Offline costs. The server precomputation in SandwichPIR consists of computing the SimplePIR hint matrix H (Fig. 2) and computing the preprocessed packing matrices $\hat{U}_1, \dots, \hat{U}_K$ for the InsPIRe packing procedure (Section 3.1). On a single GPU, computing the SimplePIR hint runs at 26–38 GB/s, and computing the packing matrices takes a

fixed ~ 0.5 s at 32 KB records. Preprocessing a 16 GB database takes under a second on a single GPU. We refer to Fig. 4 as well as Table 5 in Appendix B for further breakdowns.

Client computation. Client costs in SandwichPIR are modest. For the parameter settings we consider, query generation takes 6.3–114.1 ms and response decoding takes around 9 ms for databases ranging from 0.25 to 16 GB. We refer to Table 6 in Appendix B for the full breakdown.

Cross-client batching. As discussed in Section 3.2, batching queries from different clients allows us to take full advantage of GPU compute. In Fig. 3, we show the effective server throughput as a function of the batch size B , where we define the effective throughput to be $B \cdot |\text{db}|/T$ and T is the time to process B queries over the database db . Cross-client batching allows us to increase the effective server throughput from 488 GB/s at $B = 1$ to 24.5 TB/s at $B = 512$, an improvement of 51.5 $\times$. At the peak throughput of roughly 24.5 TB/s, the server requires an amortized 0.16 ms per query. The GPIR scheme [42] achieves a more modest 3.8 $\times$ improvement from cross-client batching on a single GPU; we conjecture that this is due to the more complex homomorphic encryption operations needed by their scheme. On a 4 GB database, their work reports an effective throughput of 1 TB/s, which is considerably lower than the 24.5 TB/s we demonstrate.

The GPU-based implementation of SimplePIR from [50] also supports cross-client batching, but due to the implementation using int32 CUDA kernels instead of tensor cores, their effective throughput saturates at 2.1 TB/s. The gap between their implementation and SandwichPIR increases from 4 $\times$ without batching to 11.9 $\times$ at the peak. Cost-wise, batching reduces SandwichPIR’s per-query compute cost

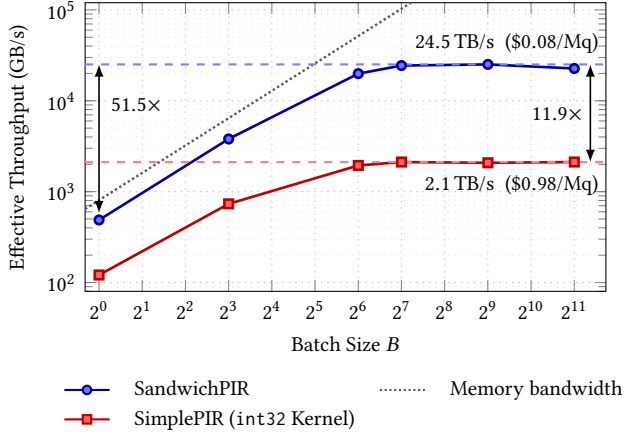


Figure 3: Effective server throughput for answering a batch of B queries over a 4 GB database with 32 KB records for SandwichPIR and the implementation of SimplePIR on the GPU from [50] (using `int32` kernels). All experiments use a single Nvidia L40S. We also show the hypothetical effective throughput achievable based on the GPU’s memory bandwidth (805 GB/s). We report the peak effective throughput for each scheme and the compute cost per million queries. We refer to Table 2 for the communication breakdown.

from \$4.24 to \$0.08 per million queries. At this point, communication costs become the dominant factor (\$10.32 per million queries and \$3.69 if we allow clients to pre-fetch an offline hint).

We also compare against the reported measurements from the concurrent work of VPIR [44], which also targets tensor cores, and IVE [45], an accelerator that supports multiple PIR schemes, including SimplePIR. On an ASIC, IVE achieves a peak effective throughput of 23 TB/s for SimplePIR [45, Table IV]. Using tensor cores on an Nvidia H100, the VPIR system achieves a peak effective throughput of 14.5 TB/s [44, Table 3]. To compare, SandwichPIR achieves a peak effective throughput of 24.5 TB/s on the smaller Nvidia L40S. Moreover, both VPIR and SimplePIR require offline communication (VPIR requires the client to upload a 5–6 MB public key and SimplePIR requires the client to download a large hint), whereas SandwichPIR maintains high effective throughput without needing offline communication. In terms of communication, SandwichPIR is comparable to SimplePIR (see Table 2). Compared to VPIR, a common measurement reported in both works is retrieving a 32 KB record from a 1 GB database: VPIR requires 480 KB of communication while SandwichPIR requires 304 KB of communication, so the two protocols are comparable. There are other parameterizations of VPIR that reduce the communication in exchange for reduced throughput.

On a CPU, cross-client batching is less effective. The work of YPIR [63] reports a 1.4 $\times$ improvement in server through-

Table 3: Performance of SandwichPIR on a g6e.48xlarge (running 8 Nvidia L40S) on 128 GB and 256 GB databases. The record size is 1 MB. We report the time and effective server throughput needed to answer a batch of B queries. The query size is 576 KB for the 128 GB database and 1.06 MB for the 256 GB database. The download for both is 3.5 MB.

B	128 GB		256 GB	
	Time	Throughput	Time	Throughput
1	33.1 ms	3.8 TB/s	58.5 ms	4.3 TB/s
8	34.1 ms	29.4 TB/s	59.5 ms	33.6 TB/s
64	41.8 ms	191 TB/s	74.0 ms	216 TB/s
128	65.6 ms	244 TB/s	119.0 ms	269 TB/s
256	128.6 ms	249 TB/s	236.6 ms	270 TB/s

put for protocols like SimplePIR and YPIR on the CPU at $B = 4$ and no further gains beyond that point. In Section 5.1, we provide some back-of-the-envelope estimates on the maximum effective throughput achievable on the CPU based on the arithmetic throughput limits of the CPU. We estimate that using the AVX-512 VNNI instruction set, it would take at least 66 r7i.2xlarge instances (and likely many more) to achieve an effective server throughput of 24.5 TB/s (which we achieve using a single Nvidia L40S). Recall that the GPU instance is only 3.5 $\times$ more expensive with current AWS pricing.

Multi-GPU support. A single Nvidia L40S has only 48 GB of device memory, which is enough to support a 32 GB database, but going beyond this becomes challenging. To support larger databases, we partition the database across multiple GPUs. Specifically, if we have M GPUs, we let each GPU hold a $1/M$ -fraction of each record in the database. This makes our approach suitable primarily in the large-record setting where records can naturally be split into M equal-size blocks. Each GPU applies the *same* query to its shard of the database. The overall response is the concatenation of the responses from all M GPUs. The advantage of this approach is that there is no communication needed between GPUs.

In Table 3, we report the performance of SandwichPIR on a g6e.48xlarge instance with 8 Nvidia L40S. We assume a record size of 1 MB (so each GPU manages a database with 128 KB records). Each GPU can hold a 32 GB shard, so this setup supports databases up to 256 GB. Over a 256 GB database, the server answers a batch of 128 queries in 119 ms, which gives an effective throughput of 269 TB/s. Each GPU sustains 96% of the best single-GPU throughput at the same 128 KB record size (Table 7), so performance scales linearly in the number of GPUs (as does the download).

Microbenchmarks. Finally, in Fig. 4, we provide some microbenchmarks for the server’s computational costs.

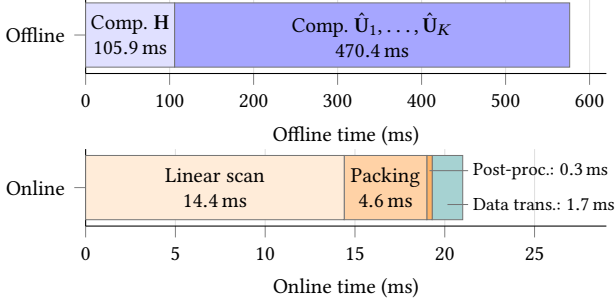


Figure 4: Breakdown of the server’s offline (i.e., Setup) and online (i.e., Answer) costs of SandwichPIR (Fig. 2) over a 4 GB database (with 32 KB records). The online costs are for processing a batch of 128 queries. For this configuration, each query is 576 KB and each response is 112 KB. Post-processing includes the summation in Eq. (3.1) and the final modulus switching step. Data transfer captures CPU/GPU communication that does *not* overlap with computation: uploading the clients’ packing keys to the GPU (1.3 ms) and downloading the response from the GPU (0.4 ms). Recall that we interleave the uploading of queries to the GPU with the packing computation (Section 4).

For the online cost, the linear scan comprises 70% of the end-to-end online processing time (14.4 of 21.0 ms). This is expected given that the linear scan must go over the full database, whereas the cost of the ring packing scales with the number of *columns* in the database matrix (multiplied by the ring dimension $n = 2^{11}$). Specifically, to process B queries over a database with ℓ_1 rows and ℓ_2 columns, the high-order term in the linear scan is performing $4B\ell_1\ell_2$ int8 arithmetic operations while that in the ring packing step is performing $16B\ell_2nt$ int8 operations. For our parameters, $n = 2^{11}$ and $t = 4$, whereas $\ell_1 = 2^{17}$.

5.1 Hardware Ceilings

In this section, we take a deeper dive to understand the throughput limits for a SimplePIR-based protocol on a CPU and on a GPU. For simplicity, we consider an instantiation of SimplePIR, where for every 8 bits of database, the server performs a single 32-bit multiplication and a single 32-bit addition. This is the setting we use throughout this work, and closely matches the setting proposed in the original SimplePIR paper [40].³

We consider the setting with cross-client batching where a server processes B queries at a time. In this case, for each byte of database, the server performs B 32-bit multiplications and B 32-bit additions. For small batch sizes (e.g., $B = 1$), the throughput is limited by the memory bandwidth of the

device whereas for very large batch sizes, throughput will be determined by the peak integer throughput of the device. We examine both limits for the hardware devices we consider in this work. Both matrix multiplications in SandwichPIR share their large operand (the database and the packing matrices, respectively) across every query in the batch, so batching raises their arithmetic intensity and moves the server from the memory-bound regime into the compute-bound one, in the sense of the roofline model [81]. Fig. 3 illustrates this roofline in action: throughput climbs along the memory-bandwidth roof before flattening at the compute ceiling. A similar observation appears in the concurrent work of [42].

The memory-bandwidth limit. On our CPU instance (an AWS r7i.2xlarge with 4 physical Sapphire Rapids cores and 8 vCPUs), the STREAM benchmark [60] sustains a Triad⁴ rate of 13.7 GB/s on a single thread and 52.8 GB/s across all 8 threads. The memory bandwidth represents a hard limit to the single-query throughput of *any* PIR scheme that must take a linear scan over the database (i.e., all of the schemes we consider in this work). On our GPU instance (an AWS g6e.xlarge with a single Nvidia L40S), the memory bandwidth is 805 GB/s. The GPU instance thus has 15× higher memory bandwidth than the CPU instance, while only being 3.5× more expensive under current AWS pricing.

Among the different single-server PIR schemes, SimplePIR comes closest to the memory-bandwidth limit. From Table 8 in Appendix B, SimplePIR on the CPU achieves a throughput of 8.65 GB/s on a 1 GB database and achieves 63% of the single-thread STREAM rate on a single core. This is because SimplePIR just performs the linear scan and nothing more. The other schemes all perform some type of hint compression and as such, fall short of the memory-bandwidth limit. SandwichPIR achieves a maximum single-query throughput of 584 GB/s over a 16 GB database, which is 72% of the memory bandwidth of the device. Given that the fastest CPU-based and GPU-based PIR schemes (that perform a linear scan) already achieve a throughput that is a significant fraction of the memory bandwidth, further scalability (beyond say a factor of 2×) must come from cross-client batching.

The integer-throughput limit. If we take a closer look at the SimplePIR protocol, each step in the linear scan consists of taking an 8-bit integer (from the database), scaling it by a 32-bit integer (from the query), and then accumulating it as a 32-bit integer. All the operations are performed modulo 2^{32} . Following our approach in Section 4, each step decomposes into exactly 4 int8 multiplications (with accumulation in a 32-bit register). Thus, we can also derive a limit on the maximum throughput for the linear scan by bounding the number of int8 operations the processor can sustain.

³The original paper [40] considered a slightly larger plaintext space, which allows 8–10 bits of the database to be processed at the cost of a single 32-bit multiplication and 32-bit addition.

⁴The STREAM Triad benchmark measures the memory bandwidth of a system by performing a fused multiply-add operation over a large array.

For instance, using the AVX-512 VNNI instruction set, each Sapphire Rapids core on the r7i.2xlarge can perform two 512-bit vpdmbusd instructions per cycle [1]. Each instruction performs 64 int8 multiply-accumulate operations (a 512-bit register holds 64 bytes, which are accumulated four at a time into 16 int32 lanes). At a clock speed of 3.2 GHz, across 4 cores, and 2 vpdmbusd instructions per cycle, this gives a bound of

$$T_{\text{CPU}}^{\max} = 4 \cdot 3.2 \text{ GHz} \cdot 2 \cdot 64 \cdot 2 = 3.3 \cdot 10^{12} \text{ ops/s} = 3.3 \text{ TOPS}.$$

Here, we are following the standard convention of counting each multiply-accumulate operation as two individual operations. In comparison, the L40S is rated at 733 int8 TOPS, which is 224 $\times$ the peak int8 throughput of our CPU instance. If we translate this into bounds on the maximum throughput for the SimplePIR linear scan, we divide by 8, since each database byte costs four int8 multiply-accumulate operations, and thus, eight int8 operations. On the CPU, the effective bound would then be

$$T_{\text{CPU}}^{\max}/8 = 409.6 \cdot 10^9 \text{ B/s} \approx 381 \text{ GB/s}. \quad (5.1)$$

Observe that this is just 7.2 $\times$ higher than the maximum single-query throughput imposed by the memory bandwidth limit.

Conversely, on the GPU, the maximum achievable throughput is 83 TB/s, which is 106 $\times$ higher than the maximum single-query throughput on the same GPU. Indeed, our cross-client batching experiments from Fig. 3 show that cross-client batching on the GPU can increase the effective throughput by a factor 51.5 $\times$ to roughly 24.5 TB/s on a single GPU (roughly 30% of the theoretical limit). We would need 66 r7i.2xlarge instances running at their (theoretical) integer-throughput limit (Eq. (5.1)) to achieve an effective throughput of 24.5 TB/s. Based on the current pricing, this is 19 $\times$ the hourly cost of the single GPU (recall that a single GPU instance is just 3.5 $\times$ more expensive than the CPU instance). In practice, the trade-off is even more favorable for the GPU since we are comparing a measured throughput to a hypothetical limit (which no scheme currently attains), and moreover, our measured GPU throughput also includes the cost of ring packing. Thus, with cross-client batching, the GPU enables a significant reduction in cost relative to the CPU instance.

We do note here that the new Intel AMX tiles on the Sapphire Rapids processor could raise the CPU’s figure by roughly 8 $\times$, to a maximum of 26.2 TOPS. On the one hand, this is still 28 $\times$ below the throughput of a single L40S (and we would still need over 8 r7i.2xlarge instances to match the measured effective throughput of SandwichPIR on the GPU). On the other hand, this could be an interesting direction to explore for improving the throughput of CPU-based PIR schemes in settings where GPU acceleration is not available. To our knowledge, no

CPU-based implementation of PIR leverages AMX at the time of writing, and we believe this warrants further study. The optimizations we developed for SandwichPIR should be very amenable to the AMX instruction set (which has a similar architecture to the tensor cores on Nvidia GPUs).

Choice of hardware acceleration. Once the server’s computation reduces to int8 operations, the preferred choice of hardware to implement SandwichPIR boils down to selecting the platform that offers the most int8 throughput per dollar. In Table 4, we list the specifications of different hardware currently available on AWS. The L40S leads at 394 int8 TOPS-hour/dollar.

5.2 An Application to Private Wikipedia

We now show how SandwichPIR can be used to support a private Wikipedia application. We use the November 2023 text-only Wikipedia snapshot from the HuggingFace release [79], which contains 6.4 million articles. After brotli compression [3], the snapshot fits into an 8 GB database with $\ell_1 = 2^{16}$ rows, each containing $\ell_2 = 2^{17}$ bytes (128 KB), and every article fits within a single record. Since the average compressed article is only 1–2 KB, we consolidate multiple articles into each logical database record. In this proof of concept, we assume the client knows the mapping from articles to the associated database record; in an actual deployment, we could substitute this with a keyword PIR [24, 5, 73, 17, 37]. With this setup, the server preprocessing takes 1.7 s on an Nvidia L40S, which is sufficient to support near real-time updates (e.g., Wikipedia receives an average of 2 edits per second [80]).

Batching. Batching client queries allows us to amortize the server cost at the expense of client latency (due to requests needing to queue up before processing). For our deployment, we consider a batch size of $B = 64$, which is the smallest batch size that maximizes the effective throughput of our protocol on a single Nvidia L40S. For English Wikipedia, which receives an average of 4.5 page views/ms [80], we expect this would introduce an average latency of under 20 ms.

Performance. On a g6e.xlarge instance with a single Nvidia L40S, the server can answer a batch of $B = 64$ queries in 25.1 ms. This translates to an amortized runtime of 0.39 ms per query and an overall throughput of over 2,500 queries per second. Based on current AWS pricing, this corresponds to a computational cost of \$0.20 per million queries.

To fetch a record, the client uploads a 320 KB query and downloads a 448 KB response, of which 288 KB is a query-independent mask that the client can cache and reuse across queries (Remark 3.2). Then, subsequent queries only need to download a 160 KB body. Under the AWS communication

Table 4: Specifications for different hardware platforms based on vendor datasheets [66]. For each device, we report the peak int8 throughput, the memory bandwidth, the AWS on-demand price, and the int8 throughput per dollar. The last two rows correspond to the CPU instance we use for benchmarking (r7i.2xlarge). Here, we calculate the peak int8 throughput using the AVX-512 VNNI and the AMX instructions at the base clock speed of 3.2 GHz, and we measured the memory bandwidth using STREAM Triad [60]. For each instance, we also indicate the accelerator (respectively, the instruction set) that provides the int8 throughput.

Instance		int8 (TOPS)	Bandwidth (GB/s)	Cost (per hour)	TOPS-hour/\$
g4dn.xlarge	Nvidia T4	130	298	\$0.526	247
g6.xlarge	Nvidia L4	242	279	\$0.805	301
g6e.xlarge	Nvidia L40S	733	805	\$1.861	394
r7i.2xlarge	AVX-512 VNNI	3.3	53	\$0.529	6.2
r7i.2xlarge	AMX	26.2	53	\$0.529	50

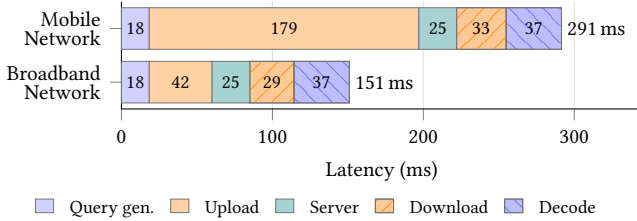


Figure 5: Estimated latency breakdown for a private Wikipedia deployment over two different network configurations [69]: a mobile network (112.07 Mbps download / 14.68 Mbps upload) and a broadband network (125.59 Mbps download / 63.08 Mbps upload). We report the client query-generation time, the server computation time, and the client decoding time, as well as the upload and download time based on the network bandwidth (excluding network latency).

model, this is \$41.29 per million queries if every query downloads the full response, and \$14.75 with caching.

On the client (measured on the g6e.xlarge host CPU, the functional equivalent of an m6a.xlarge instance), query generation takes 18.3 ms and response decoding takes 36.7 ms. Based on average network profiles for mobile and broadband networks [69], this puts the end-to-end latency at an estimated 291 ms on a mobile network and an estimated 151 ms over a fixed broadband connection. We show the breakdown in Fig. 5. We compare this to the Spiral system [62], which reported an end-to-end query latency of 4.3 s on an *uncompressed* Wikipedia variant (31 GB with 30 KB articles). Even if we optimistically assume a factor of 4× speedup if we were to apply Spiral to a compressed 8 GB database, the end-to-end latency for Spiral is over a second, and moreover, Spiral requires the server to maintain a client-specific hint. SandwichPIR reduces the latency to under 300 ms and does not require client-specific state.

6 Related Work

Starting with XPIR [61], many works have proposed concretely-efficient single-server PIR schemes using lattice-based homomorphic encryption techniques. These approaches can be categorized into three main types: schemes that require the server to maintain per-client state [6, 2, 5, 65, 62, 15, 20], schemes that require the client to download a hint in the offline phase [40, 28], and schemes that require no offline communication between the client and the server [39, 51, 63, 57, 59, 55]. Schemes that do not require any offline communication are easier to deploy, and are the focus of this work. We refer to the survey of [10] for a comprehensive overview of PIR.

Hardware acceleration for PIR. As mentioned in Section 1, a number of works have shown how to leverage hardware acceleration techniques for lattice-based homomorphic encryption to accelerate single-server PIR [71, 50, 78, 42, 45, 44]. Another line of work has explored leveraging in-storage computing for PIR [53, 22, 21]. Several works have also considered GPU acceleration of multi-server PIR [36, 47].

PIR with preprocessing. The main computational burden in many PIR schemes is the linear scan over the database. The work of Beimel et al. [11] shows that if we allow the server to first preprocess the database, then it is possible to achieve sublinear server computation; such schemes are said to be doubly-efficient. This has yielded schemes in the multi-server setting [11, 48, 32, 33, 41] and the single-server setting. In the single-server setting, existing doubly-efficient constructions either have astronomically high concrete costs [54, 67, 68, 82], only achieve a logarithmic saving in server compute [49], or only allow a single designated client (with a secret key) to make queries [12, 16, 18].

Another approach to reduce the server’s computational burden is *stateful PIR* [72], where the client first downloads a sequence of hints from the server in an offline phase and,

subsequently, the server can answer queries in amortized sublinear time [77, 74, 84, 34, 35]. However, these schemes also require the client to stream the *entire* database in an offline phase before it is able to start making queries, which limits their practical viability for large databases.

7 Conclusion

In this work, we showed how to leverage tensor cores on modern GPUs to greatly improve the concrete efficiency of single-server PIR. We showed for the first time how to achieve effective throughputs of over 24 TB/s on a single GPU and over 250 TB/s using a fleet of 8 GPUs. Using GPUs for PIR translates into substantial cost savings relative to CPU-based protocols, and presents an exciting direction for enabling PIR to support private access to large databases.

Acknowledgments

We thank Alexandra Henzinger and Samir Menon for helpful discussions, and Ryan Lehmkuhl for helpful pointers on the implementation from [50]. David J. Wu is supported in part by NSF CNS-2140975, CNS-2318701, a Sloan Fellowship, a Microsoft Research Faculty Fellowship, a Google Research Scholar Award, an Amazon Research Award, and a gift from the Stellar Development Foundation. This research was also supported in part by the Texas Advanced Computing Center (TACC) at The University of Texas at Austin, which provided computational resources (<http://www.tacc.utexas.edu>).

Statement on AI use. The technical ideas in this paper were developed without the use of AI. We used Claude as a coding assistant when developing and debugging the implementation and the benchmarking harness, and to automate the collection of the measurements reported in Section 5. We also used Claude for copy-editing, proofreading, and improving the presentation of the paper. The authors read over all AI-generated drafts and suggestions and made extensive edits to improve the exposition, reviewed all code, and verified the correctness and originality of all of the content in the paper.

References

- [1] Andreas Abel and Jan Reineke. uops.info: Characterizing latency, throughput, and port usage of instructions on Intel microarchitectures. *ASPLOS*, 2019.
- [2] Ishtiyaque Ahmad, Yuntian Yang, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Addra: Metadata-private voice communication over fully untrusted infrastructure. In *OSDI*, 2021.
- [3] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A general-purpose data compressor. *ACM Trans. Inf. Syst.*, 37(1), 2019.
- [4] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3), 2015. Estimator available at <https://github.com/malb/lattice-estimator>.
- [5] Asra Ali, Tancrede Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication-computation trade-offs in PIR. In *USENIX Security Symposium*, 2021.
- [6] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *IEEE S&P*, 2018.
- [7] Apple. Understanding how Live Caller ID Lookup preserves privacy. <https://developer.apple.com/documentation/identitylookup/understanding-how-live-caller-id-lookup-preserves-privacy>, 2024.
- [8] Apple Machine Learning Research. Combining machine learning and homomorphic encryption in the Apple ecosystem. <https://machinelearning.apple.com/research/homomorphic-encryption>, 2024.
- [9] Benny Applebaum, David Cash, Chris Peikert, and Amit Sahai. Fast cryptographic primitives and circular-secure encryption based on hard learning problems. In *CRYPTO*, 2009.
- [10] Pranav Shriram Arunachalam, Yue Chen, and Ling Ren. Private information retrieval: A tutorial and survey. *Cryptology ePrint Archive*, Paper 2026/1135, 2026.
- [11] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: PIR with preprocessing. In *CRYPTO*, 2000.
- [12] Elette Boyle, Yuval Ishai, Rafael Pass, and Mary Wootters. Can we access a database both locally and privately? In *TCC*, 2017.
- [13] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *ITCS*, 2012.
- [14] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In *FOCS*, 2011.
- [15] Alexander Burton, Samir Jordan Menon, and David J. Wu. Respire: High-rate PIR for databases with small records. In *ACM CCS*, 2024.

- [16] Ran Canetti, Justin Holmgren, and Silas Richelson. Towards doubly efficient private information retrieval. In *TCC*, 2017.
- [17] Sofia Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *ACM CCS*, 2024.
- [18] Caicai Chen, Yuval Ishai, Aayush Jain, Tamer Mour, Alon Rosen, and Chaoping Xing. Doubly-efficient secret-key PIR with low storage overhead. *IACR Cryptol. ePrint Arch.*, 2026, 2026.
- [19] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In *ACNS*, 2021.
- [20] Yue Chen and Ling Ren. OnionPIRv2: Efficient single-server PIR. *IACR Cryptol. ePrint Arch.*, 2025.
- [21] Zehao Chen, Zhaoyan Shen, Qian Wei, Hang Lu, and Lei Ju. Conflux: A high-performance keyword private retrieval system for dynamic datasets. In *IEEE HPCA*, 2026.
- [22] Zehao Chen, Honghui You, Qian Wei, Hang Lu, Lei Ju, and Zhaoyan Shen. SmartPIR: A private information retrieval system using computational storage devices. In *IEEE/ACM MICRO*, 2025.
- [23] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.*, 33(1), 2020.
- [24] Benny Chor, Niv Gilboa, and Moni Naor. Private information retrieval by keywords. *IACR Cryptol. ePrint Arch.*, 1998, 1998.
- [25] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *FOCS*, 1995.
- [26] Emma Connor, Artem Ignatyev, and Kevin Lewi. How Advanced Browsing Protection works in Messenger. <https://engineering.fb.com/2026/03/09/security/how-advanced-browsing-protection-works-in-messenger/>, 2026.
- [27] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of computation*, 19(90), 1965.
- [28] Alex Davidson, Gonalo Pestana, and Sofia Celi. FrodoPIR: Simple, scalable, single-server private information retrieval. *Proc. Priv. Enhancing Technol.*, 2023(1), 2023.
- [29] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, 2009.
- [30] Craig Gentry and Shai Halevi. Compressible FHE with applications to PIR. In *TCC*, 2019.
- [31] Craig Gentry, Shai Halevi, and Nigel P. Smart. Homomorphic evaluation of the AES circuit. In *CRYPTO*, 2012.
- [32] Ashrujit Ghoshal, Baitian Li, Yaohua Ma, Chenxin Dai, and Elaine Shi. Scalable multi-server private information retrieval. In *TCC*, 2025.
- [33] Ashrujit Ghoshal, Mingxun Zhou, Bo Peng, and Elaine Shi. ZELDA: efficient multi-server preprocessing PIR with unconditional security. In *IEEE S&P*, 2026.
- [34] Ashrujit Ghoshal, Mingxun Zhou, and Elaine Shi. Efficient pre-processing PIR without public-key cryptography. In *EUROCRYPT*, 2024.
- [35] Ashrujit Ghoshal, Mingxun Zhou, Elaine Shi, and Bo Peng. Pseudorandom functions with weak programming privacy and applications to private information retrieval. In *EUROCRYPT*, 2025.
- [36] Daniel G nther, Maurice Heymann, Benny Pinkas, and Thomas Schneider. GPU-accelerated PIR with client-independent preprocessing for large-scale applications. In *USENIX Security Symposium*, 2022.
- [37] Meng Hao, Weiran Liu, Liqiang Peng, Cong Zhang, Pengfei Wu, Lei Zhang, Hongwei Li, and Robert H. Deng. Practical keyword private information retrieval from key-to-index mappings. In *USENIX Security Symposium*, 2025.
- [38] Lena Heimberger, Christopher Patton, and Bas Westerbaan. Private SCT auditing, revisited. *IACR Cryptol. ePrint Arch.*, 2025, 2025.
- [39] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with Tiptoe. In *SOSP*, 2023.
- [40] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *USENIX Security Symposium*, 2023.
- [41] Alexandra Henzinger and Seyoon Ragavan. Two-server private information retrieval in sublinear time and quasilinear space. In *EUROCRYPT*, 2026.
- [42] Hyesung Ji, Hyunah Yu, Jongmin Kim, Wonseok Choi, G. Edward Suh, and Jung Ho Ahn. GPIR: enabling practical private information retrieval with GPUs. In *ACM International Conference on Supercomputing*, 2026.

- [43] Daniel Kales, Olamide Omolola, and Sebastian Ramacher. Revisiting user privacy for Certificate Transparency. In *IEEE EuroS&P*, 2019.
- [44] Jongmin Kim, Hyesung Ji, Jean-Luc Watson, Charles Gouert, G Edward Suh, and Jung Ho Ahn. VIPiR: A versatile GPU framework for integrating private information retrieval protocols. *arXiv preprint arXiv:2606.11536*, 2026.
- [45] Sangpyo Kim, Hyesung Ji, Jongmin Kim, Wonseok Choi, Jaiyoung Park, and Jung Ho Ahn. IVE: an accelerator for single-server private information retrieval using versatile processing elements. In *IEEE HPCA*, 2026.
- [46] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *FOCS*, 1997.
- [47] Maximilian Lam, Jeff Johnson, Wenjie Xiong, Kiwan Maeng, Udit Gupta, Yang Li, Liangzhen Lai, Ilias Leontiadis, Minsoo Rhu, Hsien-Hsin S. Lee, Vijay Janapa Reddi, Gu-Yeon Wei, David Brooks, and G. Edward Suh. GPU-based private information retrieval for on-device machine learning inference. In *ASPLOS*, 2024.
- [48] Arthur Lazzaretti, Zeyu Liu, Ben Fisch, Peihan Miao, and Charalampos Papamanthou. Multi-server doubly efficient PIR in the classical model and beyond. In *TCC*, 2025.
- [49] Keewoo Lee. Barely doubly-efficient SimplePIR. In *CRYPTO*, 2026.
- [50] Ryan Lehmkuhl, Alexandra Henzinger, and Henry Corrigan-Gibbs. Distributional private information retrieval. In *USENIX Security Symposium*, 2025.
- [51] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz-Wu. Hintless single-server private information retrieval. In *CRYPTO*, 2024.
- [52] Lucy Li, Bijeeta Pal, Junade Ali, Nick Sullivan, Rahul Chatterjee, and Thomas Ristenpart. Protocols for checking compromised credentials. In *ACM CCS*, 2019.
- [53] Jilan Lin, Ling Liang, Zheng Qu, Ishtiyaque Ahmad, Liu Liu, Fengbin Tu, Trinabh Gupta, Yufei Ding, and Yuan Xie. INSPIRE: in-storage private information retrieval via protocol and architecture co-design. In *ISCA*, 2022.
- [54] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. In *STOC*, 2023.
- [55] Chenyang Liu, Xukun Wang, and Zhifang Zhang. VIA: communication-efficient single-server private information retrieval. In *IEEE S&P*, 2026.
- [56] Wouter Lueks and Ian Goldberg. Sublinear scaling for multi-client private information retrieval. In *Financial Cryptography and Data Security*, 2015.
- [57] Ming Luo, Feng-Hao Liu, and Han Wang. Faster FHE-based single-server private information retrieval. In *ACM CCS*, 2024.
- [58] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *J. ACM*, 60(6), 2013.
- [59] Rasoul Akhavan Mahdavi, Sarvar Patel, Joon Young Seo, and Kevin Yeo. InsPIRe: Communication-efficient PIR with server-side preprocessing. In *IEEE S&P*, 2026.
- [60] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE TCCA Newsletter*, 1995.
- [61] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. XPIR : Private information retrieval for everyone. *Proc. Priv. Enhancing Technol.*, 2016(2), 2016.
- [62] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *IEEE S&P*, 2022.
- [63] Samir Jordan Menon and David J. Wu. YPIR: high-throughput single-server PIR with silent preprocessing. In *USENIX Security Symposium*, 2024.
- [64] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, 2012.
- [65] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. In *ACM CCS*, 2021.
- [66] NVIDIA Corporation. NVIDIA GPU datasheets and whitepapers. <https://resources.nvidia.com>. V100, T4, L4, and A100 product datasheets.
- [67] Hiroki Okada, Rachel Player, Simon Pohmann, and Christian Weinert. Towards practical doubly-efficient private information retrieval. In *Financial Cryptography and Data Security*, 2024.
- [68] Hiroki Okada, Rachel Player, Simon Pohmann, and Christian Weinert. On algebraic homomorphic encryption and its applications to doubly-efficient PIR. In *EUROCRYPT*, 2025.
- [69] Ookla. Speedtest Global Index. <https://www.speedtest.net/global-index>, June 2026. Median mobile: 112.07 Mbps down, 14.68 Mbps up, 23 ms. Median fixed broadband: 125.59 Mbps down, 63.08 Mbps up, 8 ms.

- [70] Ali Sah Özcan and Erkey Savas. HEonGPU: a GPU-based fully homomorphic encryption library 1.0. *IACR Cryptol. ePrint Arch.*, 2024, 2024.
- [71] Alişah Özcan. PIRonGPU: SealPIR on GPUs via the HEonGPU library. <https://github.com/Alisah-Ozcan/PIRonGPU>, 2025.
- [72] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Private stateful information retrieval. In *ACM CCS*, 2018.
- [73] Sarvar Patel, Joon Young Seo, and Kevin Yeo. Don’t be dense: Efficient keyword PIR for sparse databases. In *USENIX Security Symposium*, 2023.
- [74] Ling Ren, Muhammad Haris Mughees, and I Sun. Simple and practical amortized sublinear private information retrieval using dummy subsets. In *ACM CCS*, 2024.
- [75] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023.
- [76] Kurt Thomas, Jennifer Pullman, Kevin Yeo, Ananth Raghunathan, Patrick Gage Kelley, Luca Invernizzi, Borbala Benko, Tadek Pietraszek, Sarvar Patel, Dan Boneh, and Elie Bursztein. Protecting accounts from credential stuffing with password breach alerting. In *USENIX Security Symposium*, 2019.
- [77] Yinghao Wang, Jiawen Zhang, Jian Liu, and Xiaohu Yang. Crust: Verifiable and efficient private information retrieval with sublinear online time. *IACR Cryptol. ePrint Arch.*, 2023, 2023.
- [78] Zihan Wang, Lutan Zhao, Ming Luo, Zhiwei Wang, Haoqi He, Wenzhe Lv, Xuan Ding, Dan Meng, and Rui Hou. ShiftPIR: An efficient PIR system with gravity shifting from client to server. In *ACM CCS*, 2025.
- [79] Wikimedia. Wikimedia Downloads, 2023. Available at <https://huggingface.co/datasets/wikimedia/wikipedia>.
- [80] Wikimedia. Wikipedia: Statistics, 2026. <https://en.wikipedia.org/wiki/Wikipedia:Statistics>.
- [81] Samuel Williams, Andrew Waterman, and David A. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4), 2009.
- [82] Pan Xiao, Heng Zhang, Rending Ouyang, Cong Zhang, Jian Liu, Kui Ren, and Chun Chen. Towards making

doubly-efficient PIR practical. *IACR Cryptol. ePrint Arch.*, 2026, 2026.

- [83] Kevin Yeo and Sarvar Patel. Protecting your device information with private set membership. <https://security.googleblog.com/2021/10/protecting-your-device-information-with.html>, 2021.
- [84] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely simple, single-server PIR with sublinear server computation. In *IEEE S&P*, 2024.

A Correctness and Security Analysis

In this section, we give the correctness and the security analysis of the SandwichPIR protocol shown in Fig. 2.

A.1 Correctness Analysis

Throughout this section, we write $\|\mathbf{v}\|$ to denote the ℓ_∞ norm of a vector $\mathbf{v} \in \mathbb{Q}^n$ (i.e., $\|\mathbf{v}\| = \max_{i \in [n]} |v_i|$). For a polynomial $f \in \mathbb{Q}[x]/(x^n + 1)$, we write $\|f\|$ to denote the ℓ_∞ norm of the coefficient vector of the polynomial f . We start with some preliminaries on subgaussian error distributions. First, we say a random variable X is σ -subgaussian if for all $t \geq 0$,

$$\Pr[|X| > t] \leq 2 \exp(-\pi t^2 / \sigma^2).$$

The discrete Gaussian distribution $D_{\mathbb{Z}, \sigma}$ is σ -subgaussian. We will refer to σ as the parameter of the subgaussian distribution and σ^2 as the variance of the distribution. Like previous lattice-based PIR schemes (e.g., [6, 30, 65, 62, 51, 63, 20, 59, 55]), we rely on the *independence heuristic* [31, 23] when analyzing the error accumulated during homomorphic operations. Specifically, we model the (subgaussian) error terms arising in homomorphic operations as being independent. Instead of bounding the absolute magnitude of the error, we instead bound the variance σ^2 of the resulting subgaussian error distribution. As in the case of previous PIR schemes, the independence heuristic only affects correctness, and *not* security. We will also use the following standard properties on subgaussian random variables:

- **Scaling.** If X is σ -subgaussian, then cX is σ' -subgaussian where $\sigma' = |c| \sigma$.
- **Sums.** If $X_1, \dots, X_k$ are independent random variables and each X_i is σ_i -subgaussian, then $\sum_{i \in [k]} X_i$ is σ -subgaussian where $\sigma = (\sum_{i \in [k]} \sigma_i^2)^{1/2}$.
- **Bounded random variables.** If X is a B -bounded random variable (i.e., $\Pr[|X| \leq B] = 1$) with mean zero, then it is σ -subgaussian where $\sigma = \sqrt{2\pi}B$.

Finally, our analysis relies on the following (split) modulus-switching lemma from [62]:

Lemma A.1 (Modulus Switching [62, Theorem 3.4, adapted]). Let n be a power of two, let $q_{\text{out}} > q_{\text{fin},1} \geq q_{\text{fin},2} > p$, and let $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Take any $s \in R_{q_{\text{out}}}$ and any $(a, b) \in R_{q_{\text{out}}}^2$ where

$$b - s \cdot a = \lfloor q_{\text{out}}/p \rfloor \cdot \mu + e \pmod{q_{\text{out}}}$$

for some μ with $\|\mu\| \leq p/2$. Suppose the coefficients of s are independent and subgaussian with parameter σ_s , and the coefficients of e are subgaussian with parameter σ_e . Let $a' = \lfloor (q_{\text{fin},1}/q_{\text{out}}) \cdot a \rfloor \in R_{q_{\text{fin},1}}$ and $b' = \lfloor (q_{\text{fin},2}/q_{\text{out}}) \cdot b \rfloor \in R_{q_{\text{fin},2}}$. Then

$$\lfloor (q_{\text{fin},2}/q_{\text{fin},1}) \cdot (-sa') \rfloor + b' = \lfloor q_{\text{fin},2}/p \rfloor \cdot \mu + e_1 + e_2 \in R_{q_{\text{fin},2}},$$

where e_1 satisfies the deterministic bound

$$\|e_1\| \leq \frac{1}{2} \left(2 + (q_{\text{fin},2} \bmod p) + \frac{q_{\text{fin},2}}{q_{\text{out}}} (q_{\text{out}} \bmod p) \right),$$

and the coefficients of e_2 are subgaussian with variance

$$\left(\frac{q_{\text{fin},2}}{q_{\text{fin},1}} \right)^2 \frac{n\sigma_s^2}{4} + \left(\frac{q_{\text{fin},2}}{q_{\text{out}}} \right)^2 \sigma_e^2.$$

Correctness. We now state the correctness of the SandwichPIR protocol in Fig. 2:

Theorem A.2 (Correctness). Let $n, \sigma, p, q_{\text{in}}, q_{\text{out}}, q_{\text{fin},1}, q_{\text{fin},2}, z, t$ be the parameters from Fig. 2. Let ℓ_1, ℓ_2 be database parameters and $N = \ell_1 \ell_2$ be the number of database entries. We represent the database as a matrix $\mathbb{Z}_p^{\ell_1 \times \ell_2}$ whose entries are indexed by a pair (i_1, i_2) where $i_1 \in [\ell_1]$ and $i_2 \in [\ell_2]$. Then, under the independence heuristic, the PIR scheme in Fig. 2 has correctness error $\delta \leq 2 \exp(-\pi \tau^2 / \sigma_{\text{fin}}^2)$, where

$$\tau = \frac{\lfloor q_{\text{fin},2}/p \rfloor}{2} - \frac{1}{2} \left(2 + (q_{\text{fin},2} \bmod p) + \frac{q_{\text{fin},2}}{q_{\text{out}}} (q_{\text{out}} \bmod p) \right),$$

and

$$\sigma_{\text{fin}}^2 = \left(\frac{q_{\text{fin},2}}{q_{\text{fin},1}} \right)^2 \frac{n\sigma^2}{4} + \left(\frac{q_{\text{fin},2}}{q_{\text{out}}} \right)^2 (\sigma_{\text{scan}}^2 + \sigma_{\text{pack}}^2).$$

Here, σ_{scan}^2 and σ_{pack}^2 are the variances of the error introduced by the linear scan and the ring packing, respectively, and are defined as follows:

- **Linear scan:** $\sigma_{\text{scan}}^2 = \ell_1 \left(\frac{p}{2} \right)^2 \sigma^2 + \ell_1 \left(\frac{q_{\text{out}}}{q_{\text{in}}} \right)^2 \left(\frac{p}{2} \right)^2 \frac{\pi}{2} + \frac{\pi}{2}$.
- **Ring packing:** $\sigma_{\text{pack}}^2 = \frac{z^2 \sigma^2 t n^2}{4}$.

Proof. Take any security parameter λ , any database $\mathbf{D} \in \mathbb{Z}_p^{\ell_1 \times \ell_2}$, and any target index $(i_1, i_2) \in [\ell_1] \times [\ell_2]$. Let $(\text{pp}, \text{edb}) \leftarrow \text{Setup}(1^\lambda, \mathbf{D})$, $(\text{qu}, \text{st}) \leftarrow \text{Query}(\text{pp}, (i_1, i_2))$, and $\text{ans} \leftarrow \text{Answer}(\text{edb}, \text{qu})$. We bound the probability that $\text{Recover}(\text{st}, \text{ans}) \neq \mathbf{D}_{i_1, i_2}$. By construction of Fig. 2, these algorithms operate as follows:

- First, the setup algorithm samples $\mathbf{A} \in \mathbb{Z}_{q_{\text{out}}}^{n \times \ell_1}$ and $\mathbf{a}_{\text{ks}, \tau_5}, \mathbf{a}_{\text{ks}, \tau_{2n-1}} \xleftarrow{R} R_{q_{\text{out}}}^{\ell_1}$. It computes the hint matrix $\mathbf{H} = \mathbf{A}\mathbf{D} \in \mathbb{Z}_{q_{\text{out}}}^{n \times \ell_2}$ as well as the preprocessed matrices $\hat{\mathbf{U}}_1, \dots, \hat{\mathbf{U}}_K$.
- The query algorithm samples $s \leftarrow D_{R, \sigma}$ and defines the vector $\tilde{\mathbf{s}} \in \mathbb{Z}^n$ to be the coefficient vector of the polynomial s . Next, it samples $\tilde{\mathbf{e}} \leftarrow D_{\mathbb{Z}, \sigma}^{\ell_1}$ and sets

$$\mathbf{q}_0^\top = -\tilde{\mathbf{s}}^\top \mathbf{A} + \tilde{\mathbf{e}}^\top + \lfloor q_{\text{out}}/p \rfloor \mathbf{u}_{i_1}^\top \in \mathbb{Z}_{q_{\text{out}}}^{\ell_1}$$

$$\mathbf{q} = \lfloor (q_{\text{in}}/q_{\text{out}}) \mathbf{q}_0 \rfloor \in \mathbb{Z}_{q_{\text{in}}}^{\ell_1}.$$

It also computes the key-switching keys

$$\mathbf{b}_{\text{ks}, \tau}^\top = -s \cdot \mathbf{a}_{\text{ks}, \tau}^\top + \mathbf{e}_{\text{ks}, \tau}^\top + \tau(s) \mathbf{g}_z^\top$$

for each $\tau \in \{\tau_5, \tau_{2n-1}\}$. The query algorithm outputs $\text{st} = (s, i_2)$ and $\text{qu} = (\mathbf{q}, \mathbf{b}_{\text{ks}, \tau_5}, \mathbf{b}_{\text{ks}, \tau_{2n-1}})$.

- The answer algorithm starts by computing

$$\mathbf{r}_0^\top = \mathbf{q}^\top \mathbf{D} \in \mathbb{Z}_{q_{\text{in}}}^{\ell_2}$$

$$\mathbf{r}^\top = \lfloor (q_{\text{out}}/q_{\text{in}}) \mathbf{r}_0^\top \rfloor \in \mathbb{Z}_{q_{\text{out}}}^{\ell_2}.$$

In combination with the hint $\mathbf{H} = \mathbf{A}\mathbf{D} \in \mathbb{Z}_{q_{\text{out}}}^{n \times \ell_2}$, the vector $\mathbf{r}$ defines a collection of ℓ_2 scalar-LWE ciphertexts

$$\mathbf{C} = \begin{bmatrix} \mathbf{H} \\ \mathbf{r}^\top \end{bmatrix} = [\mathbf{C}_1 \mid \dots \mid \mathbf{C}_K] \in \mathbb{Z}_{q_{\text{out}}}^{(n+1) \times \ell_2},$$

where $\mathbf{C}_k \in \mathbb{Z}_{q_{\text{out}}}^{(n+1) \times n}$. Then, for each $k \in [K]$, the answer algorithm applies the InsPIRe packing algorithm (Theorem 2.2) using the procedure described in Section 3.1 and obtains the ciphertexts $(a_k, b_k) \in R_{q_{\text{out}}}^2$. Finally, it modulus switches each of these to $(a'_k, b'_k) \in R_{q_{\text{fin},1}} \times R_{q_{\text{fin},2}}$ by computing

$$a'_k = \lfloor (q_{\text{fin},1}/q_{\text{out}}) \cdot a_k \rfloor \in R_{q_{\text{fin},1}}$$

$$b'_k = \lfloor (q_{\text{fin},2}/q_{\text{out}}) \cdot b_k \rfloor \in R_{q_{\text{fin},2}},$$

and outputs $\text{ans} = ((a'_1, b'_1), \dots, (a'_K, b'_K))$.

- Finally, the recover algorithm writes $i_2 = n(k-1) + j$ where $k \in [K]$ and $j \in [n]$. Using the secret key s from $\text{st} = (s, i_2)$, it decrypts the k^{th} response ciphertext (a'_k, b'_k) by computing

$$v' = \lfloor (q_{\text{fin},2}/q_{\text{fin},1}) \cdot (-sa'_k) \rfloor + b'_k \in R_{q_{\text{fin},2}}$$

$$v = \lfloor (p/q_{\text{fin},2}) \cdot v' \rfloor \in R_p,$$

and outputs the coefficient of x^{j-1} in v .

It suffices to analyze the distribution of $v \in R_p$. We analyze each step individually:

Linear scan. By construction of $\mathbf{q}_0$, there exists a vector $\mathbf{k}_q \in \mathbb{Z}^{\ell_1}$ such that *over the integers*,

$$\mathbf{q}_0^\top = -\tilde{\mathbf{s}}^\top \mathbf{A} + \tilde{\mathbf{e}}^\top + \lfloor q_{\text{out}}/p \rfloor \mathbf{u}_{i_1}^\top - q_{\text{out}} \mathbf{k}_q^\top.$$

Since $\mathbf{q} = \lfloor (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{q}_0 \rfloor$, this means *over the rationals*, there exists $\hat{\mathbf{e}}_q \in \mathbb{Q}^{\ell_1}$ where $\|\hat{\mathbf{e}}_q\| \leq 1/2$ such that

$$\mathbf{q}^\top = (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{q}_0^\top + \hat{\mathbf{e}}_q^\top.$$

Next, by construction of $\mathbf{r}_0$, there exists a vector $\mathbf{k}_r \in \mathbb{Z}^{\ell_2}$ such that *over the integers*,

$$\mathbf{r}_0^\top = \mathbf{q}^\top \mathbf{D} - q_{\text{in}} \mathbf{k}_r^\top.$$

Since $\mathbf{r}^\top = \lfloor (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{r}_0^\top \rfloor$, this means *over the rationals*, there exists $\hat{\mathbf{e}}_r \in \mathbb{Q}^{\ell_2}$ where $\|\hat{\mathbf{e}}_r\| \leq 1/2$ such that

$$\mathbf{r}^\top = (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{r}_0^\top + \hat{\mathbf{e}}_r^\top.$$

Chaining these relations together and using the fact that $\mathbf{H} = \mathbf{A}\mathbf{D}$, the following holds *over the rationals*:

$$\begin{aligned} \mathbf{r}^\top &= (q_{\text{out}}/q_{\text{in}}) \cdot \mathbf{q}^\top \mathbf{D} - q_{\text{out}} \mathbf{k}_r^\top + \hat{\mathbf{e}}_r^\top \\ &= \mathbf{q}_0^\top \mathbf{D} + (q_{\text{out}}/q_{\text{in}}) \hat{\mathbf{e}}_q^\top \mathbf{D} - q_{\text{out}} \mathbf{k}_r^\top + \hat{\mathbf{e}}_r^\top \\ &= -\tilde{\mathbf{s}}^\top \mathbf{H} + \lfloor q_{\text{out}}/p \rfloor \mathbf{u}_{i_1}^\top \mathbf{D} + \mathbf{e}_{\text{scan}}^\top - q_{\text{out}} (\mathbf{k}_q^\top \mathbf{D} + \mathbf{k}_r^\top), \end{aligned}$$

where

$$\mathbf{e}_{\text{scan}}^\top = \tilde{\mathbf{e}}^\top \mathbf{D} + (q_{\text{out}}/q_{\text{in}}) \hat{\mathbf{e}}_q^\top \mathbf{D} + \hat{\mathbf{e}}_r^\top.$$

Taking $\mathbf{r}$ modulo q_{out} now yields

$$\mathbf{r}^\top + \tilde{\mathbf{s}}^\top \mathbf{H} = \lfloor q_{\text{out}}/p \rfloor \mathbf{u}_{i_1}^\top \mathbf{D} + \mathbf{e}_{\text{scan}}^\top \pmod{q_{\text{out}}}.$$

In other words, if $\mathbf{h}_j$ is the j^{th} column of $\mathbf{H}$ and r_j is the j^{th} element of $\mathbf{r}$, then the pair $(\mathbf{h}_j, r_j)$ is an LWE ciphertext encrypting $\mathbf{D}_{i_1, j}$ under $\tilde{\mathbf{s}}$ with scaling factor $\lfloor q_{\text{out}}/p \rfloor$ and error $e_{\text{scan}, j}$ (i.e., the j^{th} component of $\mathbf{e}_{\text{scan}}$). Consider the error term e_{scan} :

- By construction, each component of $\tilde{\mathbf{e}}$ is subgaussian with parameter σ . Moreover, $|\mathbf{D}_{i, j}| \leq p/2$. Thus, the components of $\tilde{\mathbf{e}}^\top \mathbf{D}$ are subgaussian with variance $\ell_1 (p/2)^2 \sigma^2$.
- Next, $\|\hat{\mathbf{e}}_q\|, \|\hat{\mathbf{e}}_r\| \leq 1/2$, and thus, are subgaussian with parameter $\sqrt{2\pi}/2$.⁵ This means $(q_{\text{out}}/q_{\text{in}}) \cdot \hat{\mathbf{e}}_q^\top \mathbf{D}$ is subgaussian with variance $\ell_1 (q_{\text{out}}/q_{\text{in}})^2 (p/2)^2 (\pi/2)$.

Under the independence heuristic, we conclude that the components of $\mathbf{e}_{\text{scan}}$ are subgaussian with parameter σ_{scan} where

$$\sigma_{\text{scan}}^2 \leq \ell_1 (p/2)^2 \sigma^2 + \ell_1 (q_{\text{out}}/q_{\text{in}})^2 (p/2)^2 \frac{\pi}{2} + \frac{\pi}{2}.$$

⁵The rounding errors $\hat{\mathbf{e}}_q$ and $\hat{\mathbf{e}}_r$ are deterministic functions of the ciphertexts $\mathbf{q}_0$ and $\mathbf{r}_0$. Here, we heuristically assume they have mean zero. Note that we can avoid this heuristic by using a *randomized* rounding procedure (i.e., after scaling each entry of $\mathbf{q}_0$ and $\mathbf{r}_0$ over the rationals, we round each component up with probability equal to the fractional part of the value). Using this randomized rounding procedure, the rounding error has mean exactly zero and thus can be modeled as a subgaussian random variable.

Ring packing. Write $i_2 = n(k-1) + j$. By the above analysis, the n LWE ciphertexts $\{(\mathbf{h}_{n(k-1)+i}, r_{n(k-1)+i})\}_{i \in [n]}$ encrypt the messages $\mathbf{D}_{i_1, n(k-1)+i}$ under $\tilde{\mathbf{s}}$, each with noise parameter σ_{scan} . By [Theorem 2.2](#), packing yields $(a_k, b_k) \in R_{q_{\text{out}}}^2$ where

$$b_k - s \cdot a_k = \lfloor q_{\text{out}}/p \rfloor d_k + e_{\text{pack}, k} \in R_{q_{\text{out}}},$$

where $d_k = \sum_{i \in [n]} \mathbf{D}_{i_1, n(k-1)+i} x^{i-1} \in R_p$ and each coefficient of $e_{\text{pack}, k}$ is subgaussian with variance $\sigma_{\text{scan}}^2 + \sigma_{\text{pack}}^2$ where

$$\sigma_{\text{pack}}^2 = \frac{z^2 \sigma^2 t n^2}{4}.$$

Final modulus switch. Since $a'_k = \lfloor (q_{\text{fin}, 1}/q_{\text{out}}) a_k \rfloor$ and $b'_k = \lfloor (q_{\text{fin}, 2}/q_{\text{out}}) b_k \rfloor$, we appeal to [Lemma A.1](#) to conclude that

$$\begin{aligned} v' &= \lfloor (q_{\text{fin}, 2}/q_{\text{fin}, 1}) \cdot (-s a'_k) \rfloor + b'_k \\ &= \lfloor q_{\text{fin}, 2}/p \rfloor d_k + e_{1, k} + e_{2, k} \in R_{q_{\text{fin}, 2}}, \end{aligned}$$

where

$$\|e_{1, k}\| \leq \frac{1}{2} (2 + (q_{\text{fin}, 2} \bmod p) + (q_{\text{fin}, 2}/q_{\text{out}})(q_{\text{out}} \bmod p)),$$

and $e_{2, k}$ is a polynomial whose coefficients are subgaussian with variance

$$\sigma_{\text{fin}}^2 = (q_{\text{fin}, 2}/q_{\text{fin}, 1})^2 \frac{n\sigma^2}{4} + (q_{\text{fin}, 2}/q_{\text{out}})^2 (\sigma_{\text{scan}}^2 + \sigma_{\text{pack}}^2).$$

Bounding the failure probability. Consider the coefficient of x^{j-1} in v' . As long as the magnitude of the corresponding coefficient of $e_{1, k} + e_{2, k}$ is less than $\frac{1}{2} \lfloor q_{\text{fin}, 2}/p \rfloor$, the coefficient of x^{j-1} in $\lfloor (p/q_{\text{fin}, 2}) \cdot v' \rfloor$ is precisely that of d_k , which, by construction, is $\mathbf{D}_{i_1, i_2}$. Using the bound for $e_{1, k}$ from above, correctness holds as long as the coefficient of x^{j-1} in $e_{2, k}$ is bounded by

$$\tau = \frac{\lfloor q_{\text{fin}, 2}/p \rfloor}{2} - \frac{1}{2} \left(2 + (q_{\text{fin}, 2} \bmod p) + \frac{q_{\text{fin}, 2}}{q_{\text{out}}} (q_{\text{out}} \bmod p) \right).$$

Since each coefficient in $e_{2, k}$ is σ_{fin} -subgaussian, this coefficient is bounded by τ with probability at least $1 - 2 \exp(-\pi \tau^2 / \sigma_{\text{fin}}^2)$, and correctness holds. $\square$

Remark A.3 (Correctness for Longer Records). [Theorem A.2](#) bounds the correctness error for retrieving a single $\mathbb{Z}_p$ element. In settings where a logical record consists of k elements of $\mathbb{Z}_p$ residing within the same row of the database, the correctness error is correspondingly scaled up by k (by a union bound). In this work, we typically think of $k = \ell_2$ (i.e., the client retrieves an entire row of the database).

Remark A.4 (Partial Modulus Switching). In SandwichPIR ([Fig. 2](#)), the client only applies modulus switching to the message-dependent component of the query (i.e., switching $\mathbf{q}$ from the NTT-friendly outer modulus q_{out}

to the power-of-two inner modulus q_{in}). Critically, we do *not* apply modulus switching to the message-independent component of the ciphertext (i.e., the matrix $\mathbf{A}$). Normally, in modulus switching, we scale both components. However, in our protocol, we rescale the message-dependent component back to q_{out} after the linear scan, so we are able to scale only the message-dependent component (and leave the message-independent component unchanged). This is a deliberate choice as a standard modulus switching that scales both components would introduce substantially more noise.

More specifically, if we applied modulus switching to $\mathbf{A}$, this would introduce a small rounding error to each of the ℓ_1 rows of $\mathbf{A}$. During decryption time, each of these errors is multiplied by the secret. Since the modulus switching all happens during the linear scan, this increases the variance σ_{scan}^2 by an additional $(n\ell_1 p^2 \sigma^2)/16$. This is a factor $n/4$ larger than the first term of σ_{scan}^2 in [Theorem A.2](#). For our concrete parameters ([Section 4](#) and [Table 1](#)), $n/4 = 512$, so this translates to a substantial increase in the error width. If we compute our correctness error with a database containing $\ell_1 \geq 2^{18}$ rows, the correctness error rate increases to nearly $3/4$, which is far higher than our target value of 2^{-40} .

A.2 Security Analysis

The query structure in SandwichPIR is exactly the same as that for YPIR [\[63\]](#) and InsPIRe [\[59\]](#)⁶ (i.e., consisting of a SimplePIR query and key-switching keys for automorphisms). Correspondingly, we can rely on the same underlying key-dependent RLWE assumption from these works. Here, we rely on key-dependent security because the query contains key-switching keys for the automorphisms. Circular security assumptions of this form are standard when constructing fully homomorphic encryption [\[29, 14, 13\]](#) as well as other RLWE-based PIR schemes [\[6, 2, 65, 62, 51, 63, 57, 59\]](#). We state the assumption below (adapted from [\[63, 59\]](#)) and the security theorem, and refer to the prior works [\[63, 59\]](#) for the formal proof.

Definition A.5 (Key-Dependent Pseudorandomness for RLWE Encodings [\[63, Definition A.1, adapted\]](#)). Let λ be a security parameter, $n = n(\lambda)$ be a power-of-two, $m = m(\lambda)$ be the number of samples, $q = q(\lambda)$ be the modulus, and $\sigma = \sigma(\lambda)$ be a width parameter. Let $R = \mathbb{Z}[x]/(x^n + 1)$ and $D_{R,\sigma}$ be the discrete Gaussian distribution over R with width σ (i.e., each coefficient of the polynomial is sampled independently from $D_{\mathbb{Z},\sigma}$). Let $\mathcal{F}$ be an efficiently computable set of functions from R_q to R_q . For a bit

$b \in \{0, 1\}$ and an adversary $\mathcal{A}$, let

$$W_b := \Pr \left[\mathcal{A}^{O(\cdot)}(1^\lambda, \mathbf{a}, \mathbf{t}_b) = 1 : \begin{array}{l} s \leftarrow D_{R,\sigma} \\ \mathbf{a} \xleftarrow{R} R_q^m, \mathbf{e} \leftarrow D_{R,\sigma}^m \\ \mathbf{t}_0 = \mathbf{s}\mathbf{a} + \mathbf{e}, \mathbf{t}_1 \xleftarrow{R} R_q^m \end{array} \right],$$

where the oracle O takes as input a function $f \in \mathcal{F}$, and outputs $(a, \mathbf{s}\mathbf{a} + \mathbf{e} + f(s))$, where $a \xleftarrow{R} R_q$ and $\mathbf{e} \leftarrow D_{R,\sigma}$. We say that the key-dependent pseudorandomness assumption for RLWE encodings with parameters (n, m, q, σ) holds if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, $|W_0 - W_1| = \text{negl}(\lambda)$.

As in [\[63, 59\]](#), we rely on key-dependent pseudorandomness with respect to the set of (scaled) automorphisms $\mathcal{F}_{\text{auto}}$:

$$\mathcal{F}_{\text{auto}} = \{r \mapsto k \cdot \tau_\ell(r) : k \in \mathbb{Z}_q, \ell \in \mathbb{Z}_{2n}^*\}.$$

Security. We now give the formal security theorem. We refer to [\[63, 59\]](#) for the formal proof.

Theorem A.6 (Security). *Let $n, \sigma, q_{\text{out}}, z, t$ be the SandwichPIR parameters from [Fig. 2](#) and let ℓ_1, ℓ_2 be the database dimensions. Suppose the key-dependent pseudorandomness assumption for RLWE encodings ([Definition A.5](#)) holds with parameters $(n, m, q_{\text{out}}, \sigma)$ and function family $\mathcal{F}_{\text{auto}}$, where $m = \ell_1/n$. Then, the SandwichPIR protocol in [Fig. 2](#) satisfies query privacy.*

Proof (Sketch). We refer to [\[63, Appendix C.2\]](#) and [\[59, §6\]](#) for the formal proof. At a very high level, we argue that given the public parameters, the client's query is pseudorandom (and thus, computationally hides the client's index). We argue this via a reduction to the key-dependent pseudorandomness assumption for RLWE encodings. In the reduction, the challenger uses the $m = \ell_1/n$ challenge samples to simulate the query vector $\mathbf{q}$. To simulate the key-switching keys $\mathbf{b}_{\text{ks}, \tau_5}$ and $\mathbf{b}_{\text{ks}, \tau_{2n-1}}$, the reduction queries the key-dependent pseudorandomness oracle on the functions $r \mapsto z^{i-1} \cdot \tau_\ell(r) \in \mathcal{F}_{\text{auto}}$ for $i \in [t]$ and $\ell \in \{5, 2n-1\}$. $\square$

B Additional Evaluation

In this section, we present some additional evaluation results for SandwichPIR. First, in [Table 8](#), we provide an expanded version of [Table 2](#), where we additionally compare the performance of SandwichPIR to that of previous schemes on 1 GB databases and 16 GB databases.

Offline preprocessing costs. In [Table 5](#), we report the cost of the server's preprocessing as a function of the database size. A single GPU can preprocess a 16 GB database in under a second. The hint computation runs at 26–38 GB/s and scales linearly with the database size. The cost of computing the ring packing matrices depends only on the width of the database, which we fix at 32 KB in [Table 5](#). As

⁶The InsPIRe construction [\[63\]](#) additionally includes an interpolation mechanism that trades communication for server computation. Here, we consider the most basic variant without interpolation.

Table 5: SandwichPIR offline preprocessing costs as a function of the database size. We fix the record size to 32 KB, which matches the configuration for our main measurements (Table 2). Specifically, if we view the database as a matrix $\mathbf{D} \in \mathbb{Z}_p^{\ell_1 \times \ell_2}$, we are fixing $\ell_2 = 2^{15}$. For each database size, we measure the time to compute the SimplePIR hint matrix $\mathbf{H}$ and the time it takes to compute the ring packing matrices $\hat{\mathbf{U}}_1, \dots, \hat{\mathbf{U}}_K$ in the Setup algorithm of Fig. 2. We also report the total preprocessing time and the preprocessing throughput (i.e., the ratio of the database size to the total preprocessing time).

DB Size	Time (ms)			Throughput (GB/s)
	Hint	Packing	Total	
0.25 GB	9.7	462.5	472.2	0.53
1 GB	28.6	462.1	490.7	2.04
4 GB	105.9	470.4	576.3	6.94
8 GB	209.9	477.4	687.3	11.6
16 GB	418.3	484.6	902.9	17.7

Table 6: Client-side compute costs for the query-generation (Query) and the response-decoding (Recover) algorithms in Fig. 2 as a function of the database size. Similar to Table 5, we fix the record size to 32 KB, which matches the configuration of our main measurements (Table 2).

DB Size	Query (ms)	Recover (ms)
0.25 GB	6.30	9.15
1 GB	11.42	9.14
4 GB	31.76	9.11
8 GB	59.87	9.22
16 GB	114.07	9.12

such, computing the ring packing matrices requires a steady 500 ms of compute. The throughput of computing the ring packing matrices is slower than that for hint computation because it additionally requires gadget decompositions and repeated conversions between the coefficient and the NTT representations of the underlying polynomials (Section 4). Overall, when the record size is fixed, the preprocessing throughput improves with the database size, from 0.53 to 17.7 GB/s, as the cost of computing the packing matrices is amortized over the size of the database.

Client-side costs. In Table 6, we provide a breakdown of the client’s compute costs as a function of the database size. Client timings are measured on the host CPU of the g6e.xlarge (an AMD EPYC 7R13, which is the same processor as the m6a.xlarge). The client’s costs are modest: query generation takes 32 ms on a 4 GB database, and response decoding is under 10 ms at every database size. For a database with ℓ_1 rows and ℓ_2 columns, the cost of Query is linear in

Table 7: Server throughput (GB/s) for SandwichPIR as a function of the database size and the record width ℓ_2 . Recall that we model the database as a matrix $\mathbf{D} \in \mathbb{Z}_p^{\ell_1 \times \ell_2}$ and on each query, the client recovers a row of $\mathbf{D}$. For each database size and each record width below, we measure the maximum effective server throughput (GB/s) for processing a batch of B queries. We report the best observed batch size B in parenthesis next to the effective throughput. For each database size, we highlight the cell (i.e., the record width ℓ_2) that maximizes effective server throughput.

Record Size	Database Size		
	1 GB	4 GB	16 GB
8 KB	9,313 (512)	9,329 (128)	9,231 (128)
16 KB	14,443 (512)	16,207 (512)	14,354 (64)
32 KB	14,714 (512)	25,304 (512)	21,326 (256)
64 KB	9,195 (512)	25,646 (512)	32,769 (128)
128 KB	5,189 (256)	16,804 (256)	35,739 (128)
256 KB	2,763 (256)	9,803 (256)	26,243 (128)

ℓ_1 , since the client encrypts a vector of dimension ℓ_1 . The two key-switching keys contribute a fixed (and modest) overhead (around 5 ms). The cost of Recover scales with $K = \ell_2/n$, since the client needs to perform a single polynomial multiplication for each of the K packed response ciphertexts. Since the record width is fixed, $K = 16$ at every database size, and the running time of Recover is essentially constant in Table 6.

Impact of database shape. In Table 7, we report the effective server throughput as a function of both the database size and the record size. Since the client recovers a row of $\mathbf{D}$ on each query, the record size is a free parameter. For a database with N bytes, we can partition the N bytes into ℓ_1 rows, each containing ℓ_2 bytes, where $N = \ell_1 \ell_2$. When processing a batch of B queries, the server’s linear scan always performs $O(BN)$ arithmetic operations regardless of the shape of the database. However, the query size and the cost of the ring packing both depend on the shape of the database. First, the client’s query contains ℓ_1 elements over $\mathbb{Z}_{q_{in}}$. As discussed in Section 4, the server has to decompose these elements into byte-size chunks to leverage tensor cores for the matrix multiplication. The cost of this initial step depends on the query dimension ℓ_1 . On the other hand, the ring packing takes $O(Bn\ell_2)$ operations (Section 3.1), and is linear in ℓ_2 . Since the query preprocessing time scales linearly with ℓ_1 while the ring packing time scales linearly with ℓ_2 , we would expect the server throughput to be maximized when the two dimensions are roughly equal (e.g., when the database is square). This coincides with the measurements shown in Table 7. Note that balancing ℓ_1 and ℓ_2 also roughly balances the total communication; recall that

the size of the client's query scales linearly with ℓ_1 while
the size of the server's response scales linearly with ℓ_2 .

Table 8: Performance comparison of different PIR schemes on databases of varying sizes (1 GB, 4 GB, 16 GB). We use a fixed record size of 32 KB for all experiments. For each scheme, we report the offline communication (if any), the online upload (On $\uparrow$), the online download (On $\downarrow$), the server response time, and the server throughput (ratio of database size to response time). We report the server computation and communication costs based on the AWS cost model for handling a million queries. We also report the offline bandwidth cost needed to support a million clients. We omit the 16 GB configuration for systems that triggered out-of-memory errors on our benchmarking instances (KsPIR, PIRonGPU, and SimplePIR on GPU).

	Scheme	DB	Communication			Server Time		Online Cost (per 10^6 queries)			Offline
			Off $\downarrow$	On $\uparrow$	On $\downarrow$	Time	Throughput	Compute	Bandwidth	Total	Cost
CPU	SimplePIR [40, 50]	1 GB	144 MB	128 KB	40 KB	116 ms	8.65 GB/s	\$16.99	\$3.69	\$20.68	\$13,590
		4 GB	144 MB	512 KB	40 KB	468 ms	8.55 GB/s	\$68.75	\$3.69	\$72.44	\$13,590
		16 GB	144 MB	2 MB	40 KB	2,083 ms	7.68 GB/s	\$306.27	\$3.69	\$309.96	\$13,590
	HintlessPIR [51]	1 GB	—	248 KB	2.97 MB	953 ms	1.05 GB/s	\$140.11	\$280.12	\$420.23	—
		4 GB	—	722 KB	2.97 MB	1,427 ms	2.80 GB/s	\$209.77	\$280.12	\$489.89	—
		16 GB	—	2.56 MB	2.97 MB	3,244 ms	4.93 GB/s	\$476.82	\$280.13	\$756.95	—
	YPIR [63]	1 GB	—	686 KB	120 KB	424 ms	2.36 GB/s	\$62.28	\$11.06	\$73.34	—
		4 GB	—	1.33 MB	120 KB	925 ms	4.32 GB/s	\$135.98	\$11.06	\$147.04	—
		16 GB	—	3.95 MB	120 KB	2,928 ms	5.46 GB/s	\$430.39	\$11.06	\$441.45	—
	KsPIR [57]	1 GB	—	2.31 MB	224 KB	718 ms	1.39 GB/s	\$105.57	\$20.64	\$126.21	—
		4 GB	—	2.53 MB	224 KB	2,745 ms	1.46 GB/s	\$403.45	\$20.64	\$424.09	—
	InsPIRe [59]	1 GB	—	392 KB	96 KB	318 ms	3.14 GB/s	\$46.74	\$8.85	\$55.59	—
		4 GB	—	1.04 MB	96 KB	718 ms	5.57 GB/s	\$105.54	\$8.85	\$114.39	—
		16 GB	—	3.66 MB	96 KB	2,380 ms	6.72 GB/s	\$349.87	\$8.85	\$358.72	—
GPU	PIRonGPU [71]	1 GB	—	144 KB	2.25 MB	851 ms	1.18 GB/s	\$439.86	\$212.34	\$652.20	—
		4 GB	—	144 KB	2.25 MB	3,075 ms	1.30 GB/s	\$1,589.60	\$212.34	\$1,801.94	—
	SimplePIR [40, 50] (int32)	1 GB	144 MB	128 KB	40 KB	8.23 ms	121 GB/s	\$4.25	\$3.69	\$7.94	\$13,590
		4 GB	144 MB	512 KB	40 KB	32.5 ms	123 GB/s	\$16.80	\$3.69	\$20.49	\$13,590
	SandwichPIR	1 GB	—	192 KB	112 KB	3.40 ms	294 GB/s	\$1.76	\$10.32	\$12.08	—
		4 GB	—	576 KB	112 KB	8.19 ms	488 GB/s	\$4.24	\$10.32	\$14.56	—
		16 GB	—	2.11 MB	112 KB	27.4 ms	584 GB/s	\$14.16	\$10.32	\$24.48	—
	SandwichPIR (with hint)	1 GB	72 KB	192 KB	40 KB	3.40 ms	294 GB/s	\$1.76	\$3.69	\$5.45	\$6.64
		4 GB	72 KB	576 KB	40 KB	8.19 ms	488 GB/s	\$4.24	\$3.69	\$7.93	\$6.64
		16 GB	72 KB	2.11 MB	40 KB	27.4 ms	584 GB/s	\$14.16	\$3.69	\$17.85	\$6.64